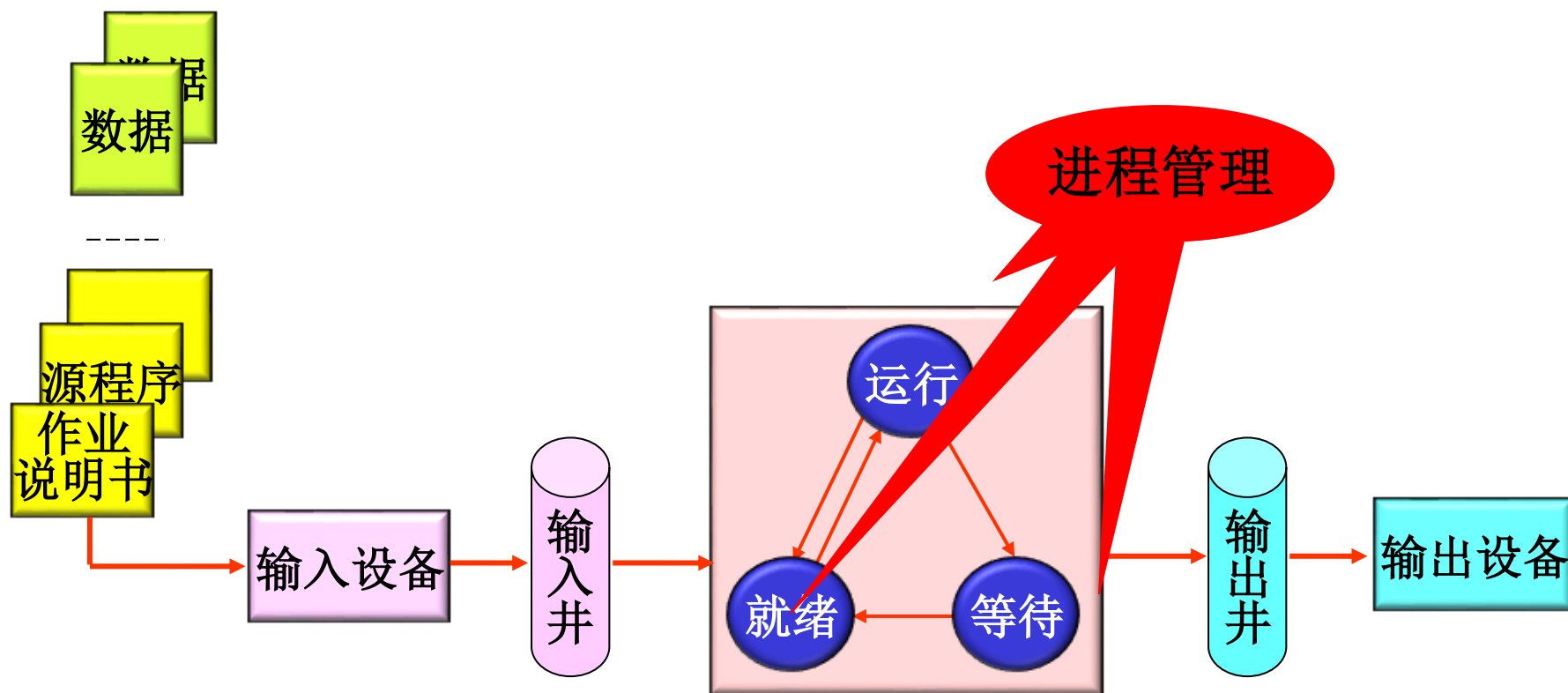
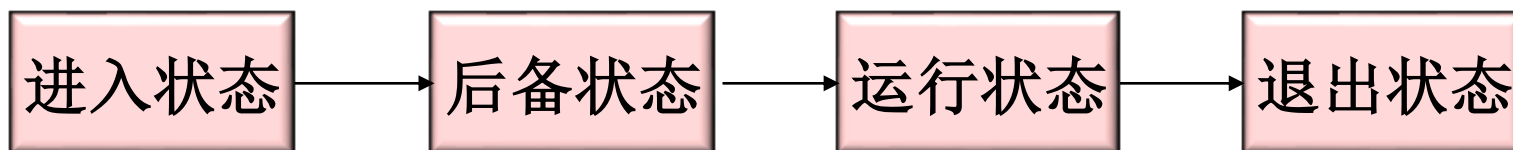






第三章 进程管理





进程管理



- 为了描述程序在并发执行时对系统资源的共享，需要一个描述程序执行时动态特征的概念，这就是进程。
- 本章将讨论进程概念、进程控制和进程间关系。

1. 概述 (第三章)
2. 进程的描述 (第三章)
3. 进程控制 (第三章)
4. 线程 (第三章)
5. 进程互斥和同步 (第四章)
6. 进程间通信 (第五章)
7. 死锁问题 (第六章)
8. 进程其他方面的举例



3.1 概述



内容提要:

1. 多道程序执行带来的问题

- ✓程序的顺序执行
- ✓程序的并发执行
- ✓带来的问题

2. 为什么需要进程，而不是程序？

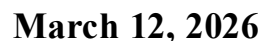
- ✓进程的引入
- ✓进程的概念



3.1.1 程序的顺序执行和并发执行

① **顺序执行**。顺序执行是单道批处理系统的执行方式，也用于简单的单片机系统

② **并发执行**。是OS的主要特征之一，现代OS多为**并发执行**。引入并发执行的目的是为了提高**资源利用率**





3.1 概述

3.1.1 程序的顺序执行和并发执行

(1) 程序的顺序执行

- **程序**：完成所要求的功能时，所应采取的顺序步骤，是执行指令的有序集合。
- **程序的顺序执行**：具有独立功能的程序**独占CPU**直至得到最终结果的过程。
- **顺序执行的特征**：
 - **顺序性**：按照程序结构所指定的次序（可能有分支或循环）
 - **封闭性**：独占全部资源，计算机的状态只由该程序的控制逻辑所决定
 - **可再现性**：初始条件相同则结果相同。



3.1 概述

3.1.1 程序的顺序执行和并发执行

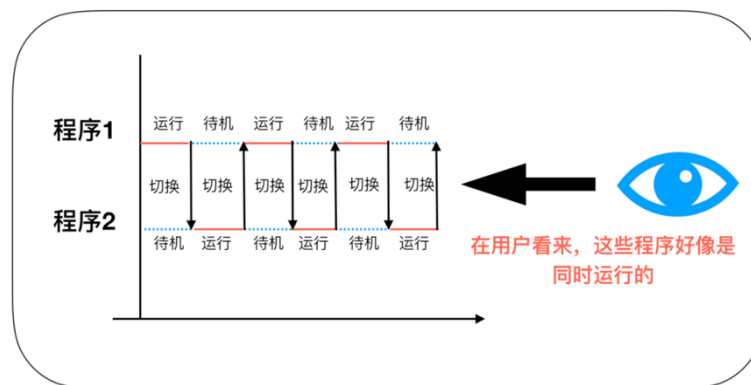
(2) 程序的并发执行

目的:

- 提高计算机的处理能力;
- 提高资源利用率

形式:

- ① **多道程序的并发执行**: 多道程序环境下的多道程序的并发执行。执行环境具有的特点: 独立性、随机性、资源共享性
- ② **一个程序中几个程序段中并发**: 在某道程序的几个程序段中并发, 包含可同时执行或可颠倒顺序执行的代码。每个并发的程序段对应一个**线程**。
- ③ **问题**: 这两种并发有何不同?





3.1 概述



3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

● **定义**：程序的并发执行是指一组在逻辑上互相**独立**的**程序或程序段**在**执行时间**上**客观上互相重叠**，即一个程序或程序段的执行尚未结束，另一个程序（段）的执行已经开始的执行方式。

➤ **并发**：在一段时间内的同时执行

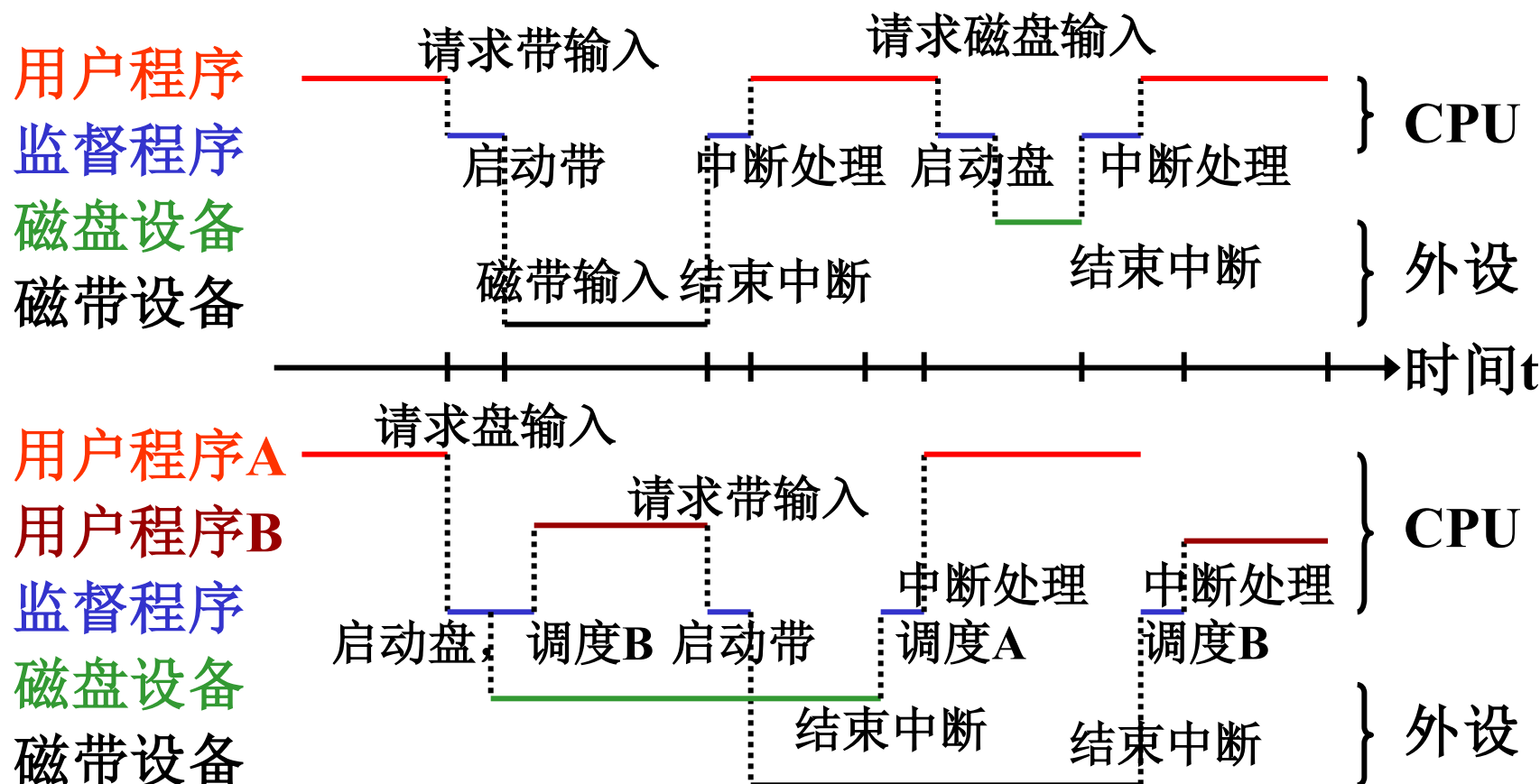
➤ **并行**：在同一物理时刻的同时执行



3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行





3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

● 并发执行的特征

- **中断(异步)性**: “走走停停”，一个程序可能走到中途停下来，失去原有的时序关系；
- **失去封闭性**: 合作，共享资源，受其他程序的控制逻辑的影响。如：一个程序写到存储器中的数据可能被另一个程序修改，失去原有的不变特征；
- **失去可再现性**: 失去封闭性→失去可再现性；外界环境在程序的两次执行期间发生变化，失去原有的可重复特征。



3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

● 不加控制的并发执行所带来的影响

➤ 想想铁路、民航售票系统

➤ 例1：两个程序都对同一个变量进行操作的情况，以下两个程序，对A变量进行读取10，P1程序将其减5，P2程序将其减8，两程序提交之后，A的状态为2，即P1程序的更新丢失了。如果A代表账户余额或者代表火车票的余额，这种错误是不可接受的。

时序	P1	P2
0	①读A=10	读A=10
1	②	
2	③A=A-5写回	A=A-8写回
3	④	



3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

- 不加控制的并发执行所带来的影响

例2[自学]: 利用堆栈管理一块内存区中各数据块的使用情况

- 用getaddr(top)从栈顶取出相应的内存块地址;
- 用reladdr(blk)将数据块的地址 (blk指向的地址) 放入堆栈中。

```
proc getaddr(top)
Begin
  local r;
  1.1  $r \leftarrow s[top]$ ;
  1.2  $top \leftarrow top+1$ ;
  1.3 return (r);
End;
```

```
Proc reladdr(blk)
Begin
  2.1  $top \leftarrow top-1$ ;
  2.2  $s[top] \leftarrow blk$ ;
End;
```



3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

proc getaddr(top) getaddr(top) 与 reladdr(blk) 的并发执行

Begin

 local r;

 1.1 $r \leftarrow s[top]$;

 1.2 $top \leftarrow top+1$;

 1.3 return (r);

End;

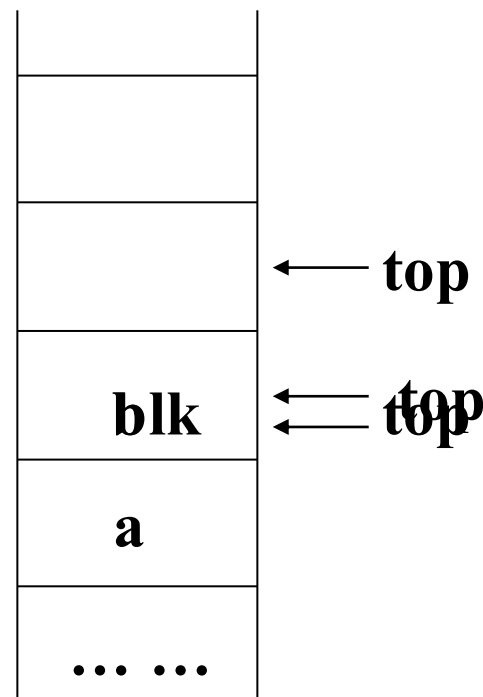
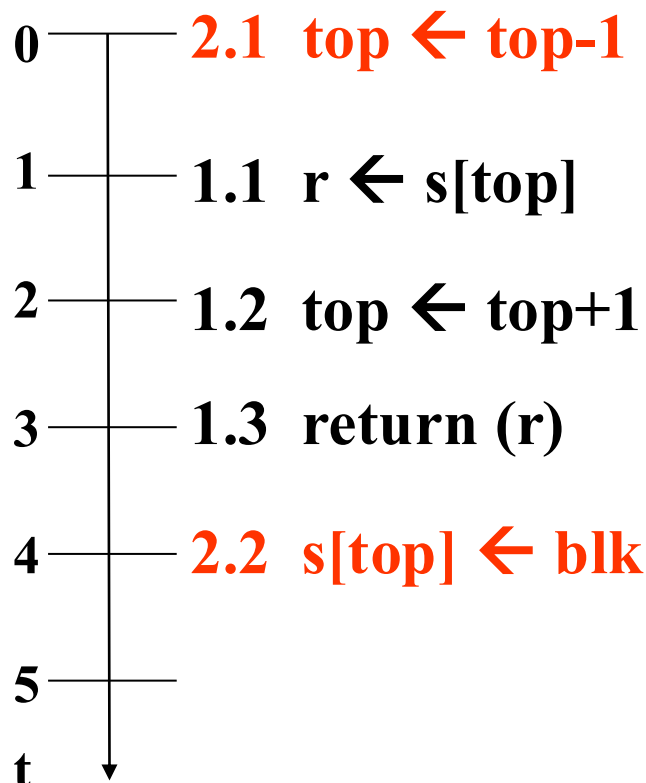
Proc reladdr(blk)

Begin

 2.1 $top \leftarrow top-1$;

 2.2 $s[top] \leftarrow blk$;

End;





3.1 概述



3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

● 例子的说明

➤ `getaddr()`和`reladdr()`的并发执行，产生了错误的结果，不同执行顺序得到不同的结果，程序的执行不再具有封闭性和结果的可再现性；

➤ 原因是什么？

● 对**公共变量**（堆栈或堆栈指针）的**共享**引起的；

➤ 为了获得结果的可再现性，程序的并发执行是**需要条件和制约的**。

● 生活中的例子：微信**私聊**、**群聊**



3.1 概述



3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

● **并发执行的条件**：达到封闭性和可再现性

✓ 并发执行失去封闭性的原因是共享资源的影响，去掉这种影响即可。

✓ 1966年，由Bernstein给出并发执行的条件（这里没有考虑执行速度的影响）。程序 $P(i)$ 针对的读变量集和写变量集为 $R(i)$ 和 $W(i)$ 。任意两个程序 $P(i)$ 和 $P(j)$ 可并发的条件：

● $R(i) \cap W(j) = \Phi$

● $W(i) \cap R(j) = \Phi$ 保证一个程序的两次读之间数据不变化

● $W(i) \cap W(j) = \Phi$ 保证写的结果不丢掉

问题是：这个条件不好检查，代价高。**怎么办？**



3.1 概述

3.1.1 程序的顺序执行和并发执行

(2) 程序的并发执行

- ❁ 程序活动不再处于一个封闭系统中，呈现了独立性、并发性、动态性以及它们之间的相互制约性。
- ❁ “程序”这个静态概念已经不能如实地反映程序活动的这些特征
- ❁ 为此，引入“进程”这一动态概念来描述系统和用户的程序活动。





3.1 概述

3.1.1 程序的顺序执行和并发执行

(3) 小结

- 多道程序的并发执行 VS 人类社会的人类活动
- 前者需要并发控制
- 后者亦是需要遵守法律法规、遵纪守法，否则社会就会混乱。例如交通和交通规则





3.1 概述

3.1.2 进程的定义

(1) 定义

进程：一个具有一定独立功能的程序在一个数据集合上的一次动态执行过程。简言之，**进程是程序的一次执行活动。**

- 进程描述了程序的动态执行过程；
- 它对应虚拟处理机、虚拟存储器和虚拟外设等资源的分配和回收；
- 反映系统中程序执行的并发性、随机性和资源共享；
- 引入多进程，提高了对硬件资源的利用率，但又带来额外的空间和时间开销，增加了OS的复杂性。



3.1 概述



3.1.2 进程的定义

(2) 进程的特征

● 动态性:

- 进程对应程序的执行
- 进程是动态产生: 创建→运行→消亡;
- 进程在其生命周期内, 在几种状态间转换;

● 独立性: 各进程的地址空间相互独立, 除非采用进程间通信手段

● 并发性: 任何进程都可以同其他进程一起向前推进

● 异步性: 每个进程都以其相对独立的不可预知的速度向前推进

● 结构化: 进程 = 代码段 + 数据段 + 进程控制块



3.1 概述

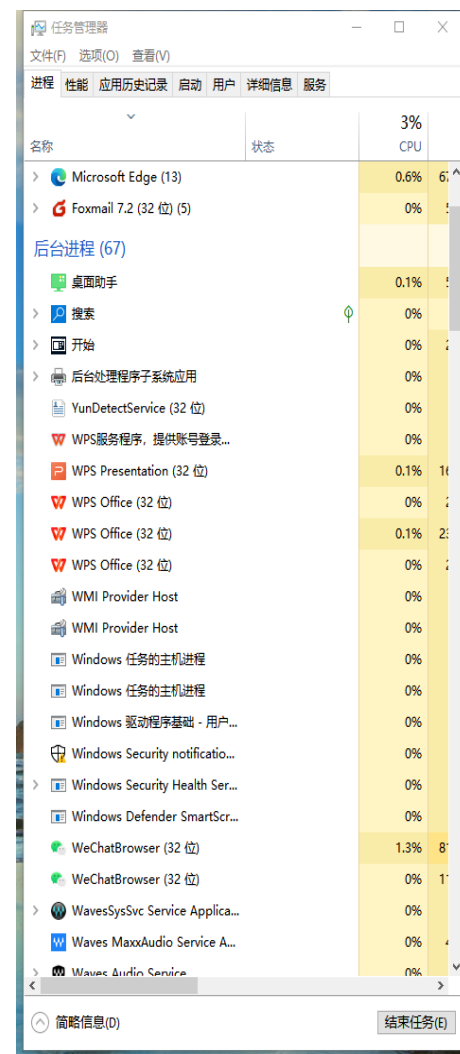
3.1.2 进程的定义

(3) 进程与程序的区别



- 进程是动态的，程序是静态的；
- 进程是暂时的，程序是永久的：进程是一个状态变化的过程，程序可长久保存；
- 组成不同：进程的组成包括程序、数据和进程控制块（即进程状态信息）；
- 对应关系：一个程序可对应多个进程(通过多次执行)；一个进程可执行多个程序(通过调用关系)；
- 进程具有并发特征，程序没有。进程具有独立性和异步性；
- 进程是竞争计算机资源的基本单位。

类比：炒菜↔菜谱，还有那样的类比？

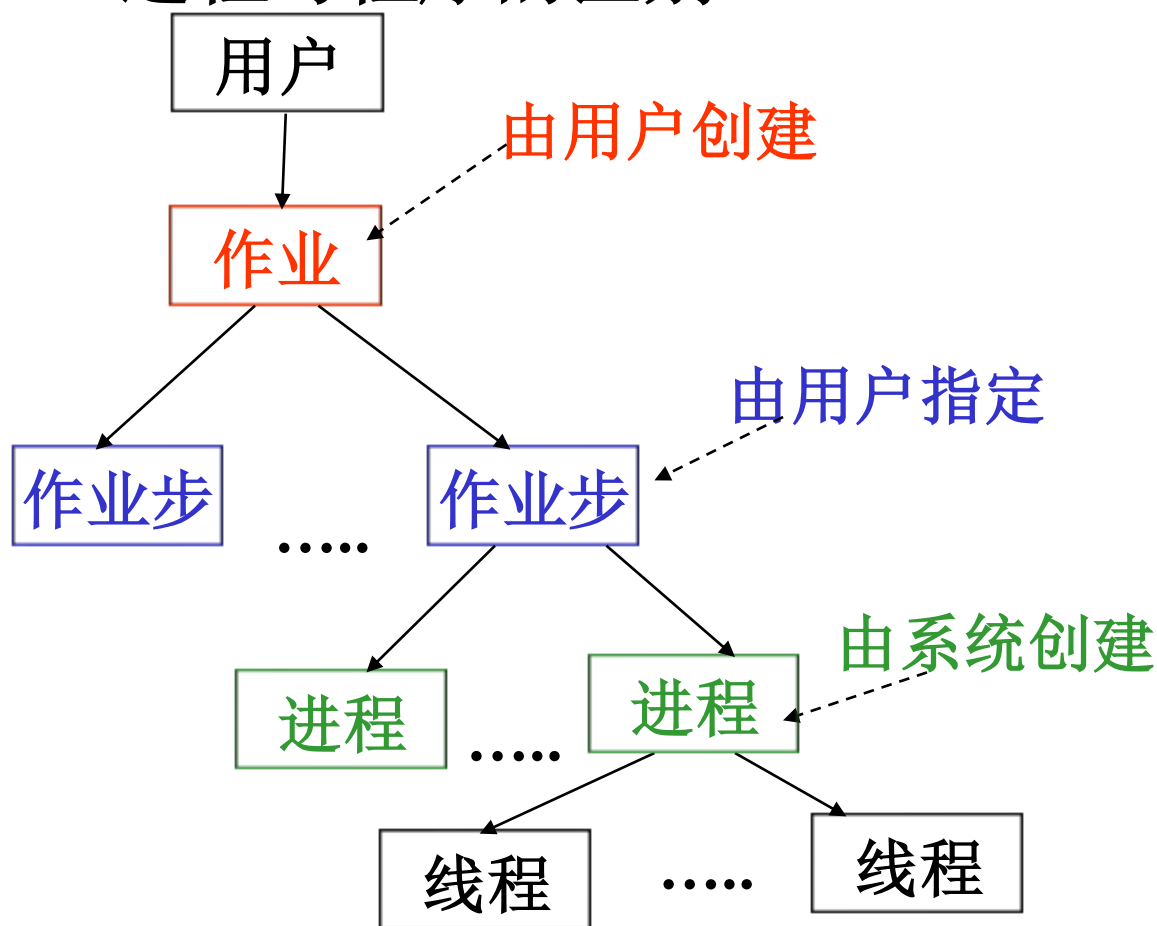




3.1 概述

3.1.2 进程的定义

(3) 进程与程序的区别





3.1 概述



- 小结
 - 程序顺序执行、特征
 - 程序并发执行、特征、带来的问题
 - 进程的引入：描述程序的动态执行
 - 进程定义、特征
 - 进程与程序的比较



- 系统调用（SC）

- 分类、POSIX标准
- 实现过程：陷入内核、保护现场、执行服务例程、恢复现场
- SC、API（Libc库函数）、应用程序、服务例程
- Linux(openEuler)的系统调用实现

- 进程概述

- 程序顺序执行、特征：顺序性、封闭性、结果可再现性
- 程序并发执行、特征、带来的问题：间断性、失去封闭性、失去可再现性
- 进程的引入：描述程序的动态执行
- 进程定义
- 进程特征：动态性、独立性、并发性、异步性、结构化
- 进程与程序的比较



3.2 进程的描述



要回答的问题：

1. 进程都包括什么？
2. 如何描述进程？ -->PCB
3. PCB是什么？作用是什么？
4. 如何组织进程（PCB如何组织）？
5. 进程有哪些状态？
6. 这些状态间如何转换？



3.2 进程的描述

3.2.1 进程的组成

进程 = 程序 + 数据 + 进程控制块PCB

- 程序是进程的不可缺少的组成部分；如果一个程序段允许被共享，则它应该是可重入的，或纯代码段；
- 数据是进程处理的对象；
- 进程控制块（Process Control Block, PCB）是进程的控制结构，是进程的唯一标识，系统通过PCB管理和控制进程。都包含什么信息？



3.2 进程的描述

3.2.2 进程控制块PCB (Process Control Block)

- PCB是由OS维护的，用来记录进程相关信息，和管理进程而设置的一个专门的数据结构
 - 包含了进程的**描述信息**、**控制信息**、**资源信息**以及**现场保护区**。
 - **想一想：PCB采用什么数据结构描述？**
- PCB是进程动态特性的集中反映
 - 系统通过PCB感知进程的存在
 - PCB中所包含的各项变量的变化
 - 通过它掌握进程的状态，达到控制进程活动的目的



3.2 进程的描述

3.2.2 进程控制块PCB (Process Control Block)

- PCB结构的全部或部分常驻内存；例如，
 - UNIX的进程的PCB由 `proc` 结构体和 `user` 结构体管理；
 - Linux(openEuler)的PCB，也叫进程描述符`struct task_struct`
- PCB随进程的创建而填写，随进程撤消而释放；
- 系统利用PCB来控制和管理进程，所以PCB是系统感知进程存在的唯一标志；
- 进程与PCB是一一对应的。



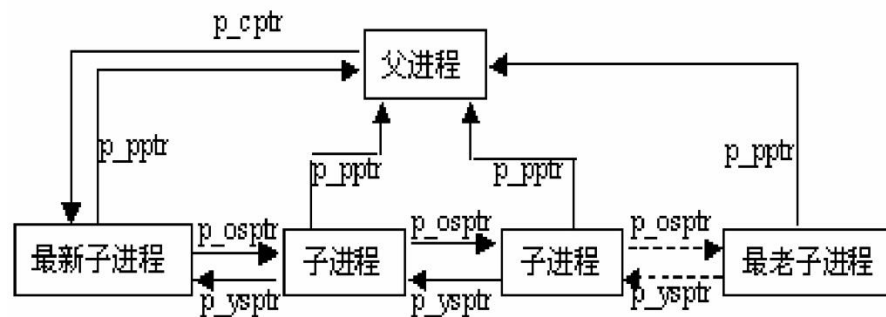
3.2 进程的描述

3.2.2 进程控制块PCB (Process Control Block)

● 进程控制块的内容 (1/3)

● 进程描述信息:

- ✓ 进程标识符 (process ID), 唯一, 通常是一个整数
- ✓ 进程名, 通常基于可执行文件名
- ✓ 用户标识符 (user ID)
- ✓ 进程组关系 (process group)





3.2 进程的描述

3.2.2 进程控制块PCB (Process Control Block)

● 进程控制块的内容 (2/3)

● 进程控制信息:

- ✓ 当前状态 (State)
- ✓ 优先级 (priority)
- ✓ 代码执行入口地址
- ✓ 程序的外存地址
- ✓ 运行统计信息 (执行时间、页面调度)
- ✓ 进程间同步和通信, 阻塞原因



3.2 进程的描述

3.2.2 进程控制块PCB (Process Control Block)

● 进程控制块的内容 (3/3)

➤ 资源占用信息:

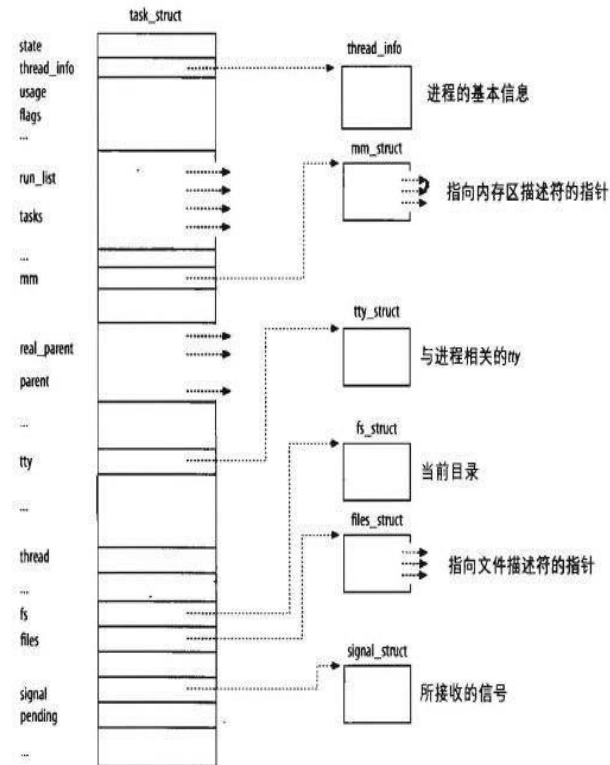
虚拟地址空间的现状、
打开文件列表

➤ CPU现场保护结构:

寄存器值（通用、程序
计数器PC、状态PSW，
地址包括栈指针）

● Linux的进程描述符结构

➤ 见[Linux的进程描述符结构sched.h](http://linux.dorland.com/linux/sched.h)





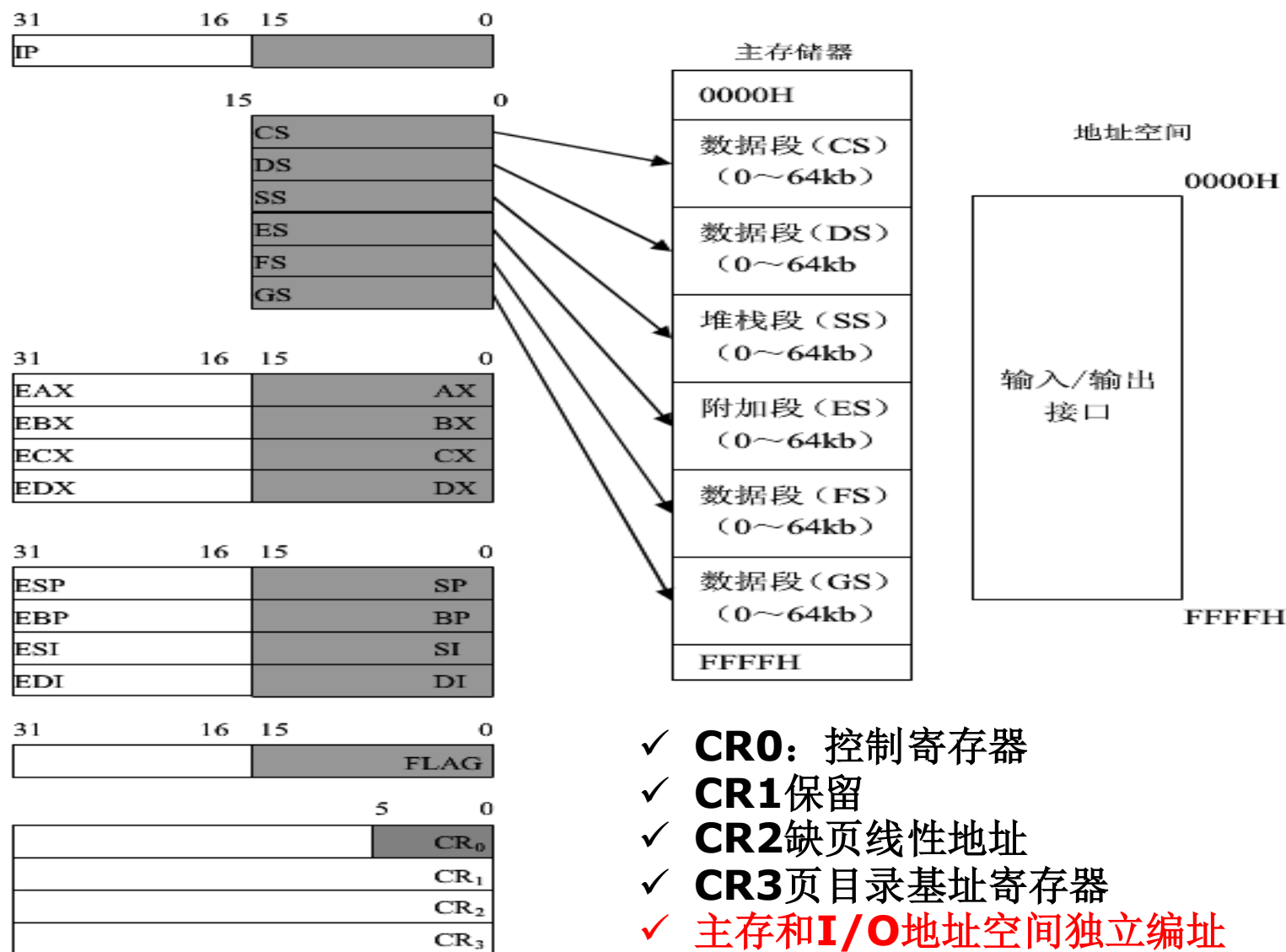
3.2 进程的描述

3.2.3 进程上下文 (Process Context)

- **进程上下文**是对**进程执行活动全过程**的**静态描述**，由进程的**用户地址空间内容**、**硬件寄存器内容**及与该进程相关的**核心数据结构**组成（PCB、核心栈、...）。
- **进程上下文**包括：
 - **用户级上下文**：进程的用户地址空间（包括用户栈各层次），包括用户正文段、用户数据段和用户栈；
 - **寄存器级上下文**：程序寄存器、处理机状态寄存器、栈指针、通用寄存器的值；
 - **系统级上下文**：
 - ✓ 静态部分：PCB和资源表格；
 - ✓ 动态部分：核心栈（核心运行使用的栈结构，不同进程在调用相同核心过程时有不同核心栈）。



I386微处理器的寄存器软件模型



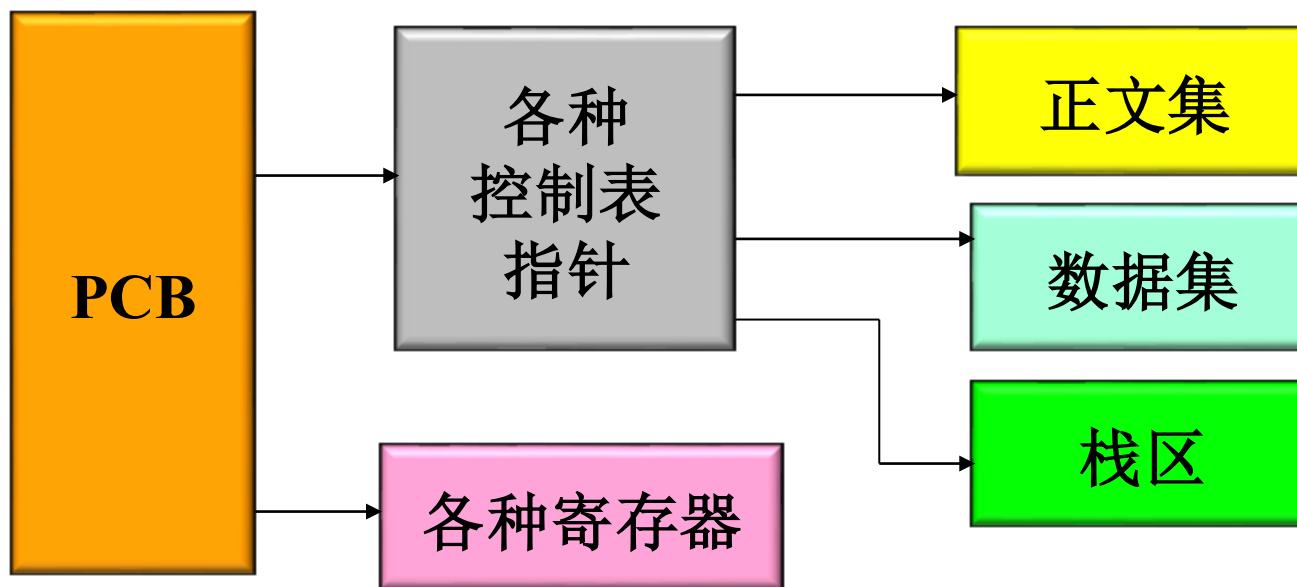


3.2 进程的描述



3.2.3 进程上下文

● 进程上下文结构





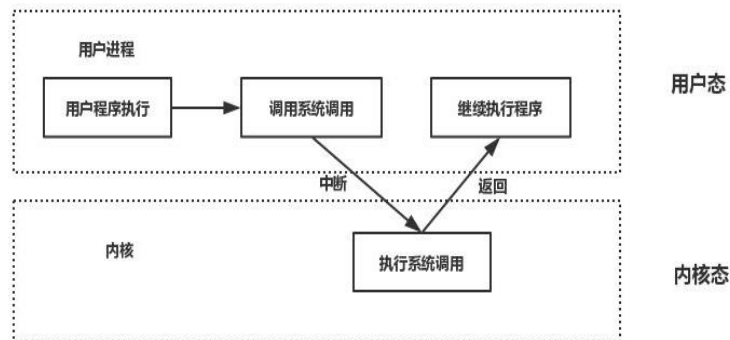
3.2 进程的描述



3.2.4 进程的状态及其转换

➤ CPU的两种运行状态

- CPU执行状态分为**用户态(User Mode)**和**内核态(Kernel M.)**
- **用户态**，也称为非特权态、目态，只能运行CPU非特权指令。进程的用户程序段执行时，CPU处于用户态。用户态时不可直接访问受保护的OS代码；如何访问呢？
- **内核态**，也称核心态、系统态、特权态、管态，可以运行CPU任何指令。此时CPU运行在OS核心代码上。核心态时可以执行OS代码，可以访问全部进程空间。如，执行用户程序调用的系统调用时，或执行中断处理程序时
- 划分用户态和系统态的原因：
 - 有利于程序的保护和共享
 - 增加了系统复杂度和系统开销

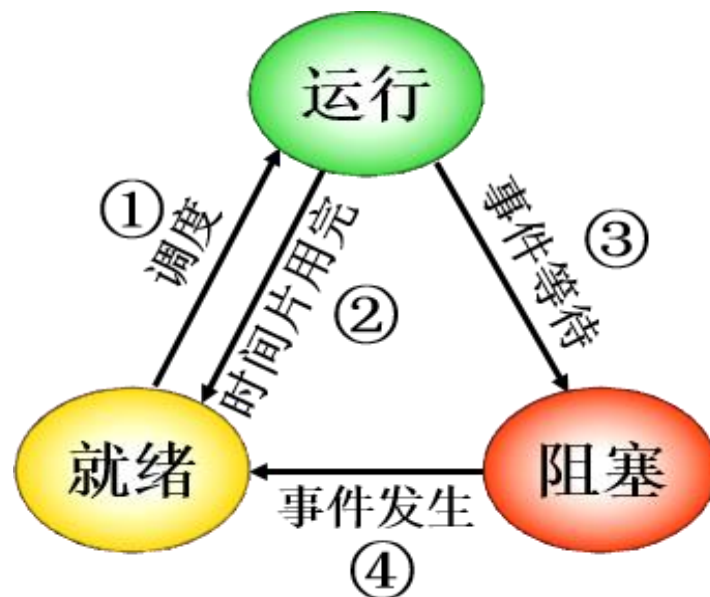




3.2 进程的描述

3.2.4 进程的状态及其转换——进程的三种基本状态

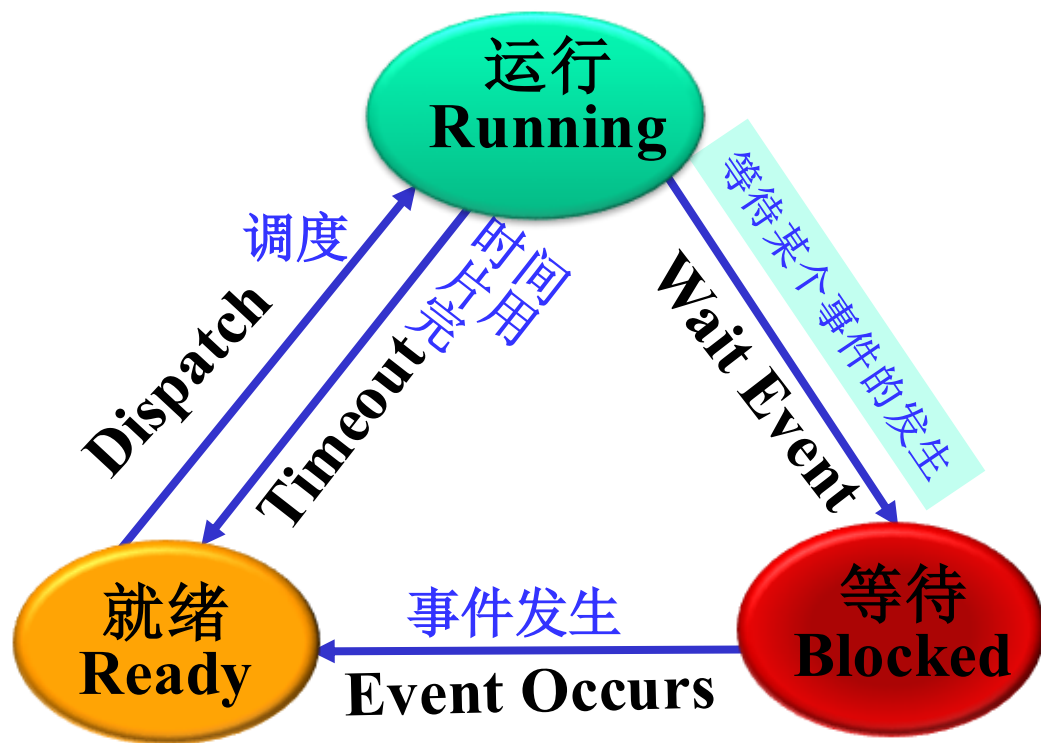
- **就绪态 (Ready)**：一个进程已经具备运行条件，但由于无CPU暂时不能运行的状态。当调度给其CPU时，立即可以运行。位于“就绪队列”中。
- **执行态 (Running)**：进程占有了包括CPU在内的全部所需资源，并在CPU上运行
- **等待态 (Blocked)**：阻塞态、挂起态、封锁态、冻结态、睡眠态。指进程因等待某种事件的发生而暂时不能运行的状态（即使CPU空闲，该进程也不可运行）。位于“等待队列”中。





3.2 进程的描述

3.2.4 进程的状态及其转换——三种基本状态及其转换



- **就绪→运行**: 调度程序选择一个新的进程运行
- **运行→就绪**:
 - 运行进程用完了时间片
 - 运行进程被中断, 因某高优先级进程处于就绪状态
- **运行→阻塞**: 当进程等待某一事件的发生时, 如:
 - 请求系统服务
 - 无新工作可做
 - 初始化I/O且必须等待结果
 - 等待某一进程提供输入 (进程间通信)
- **阻塞→就绪**: 当所等待的事件发生时

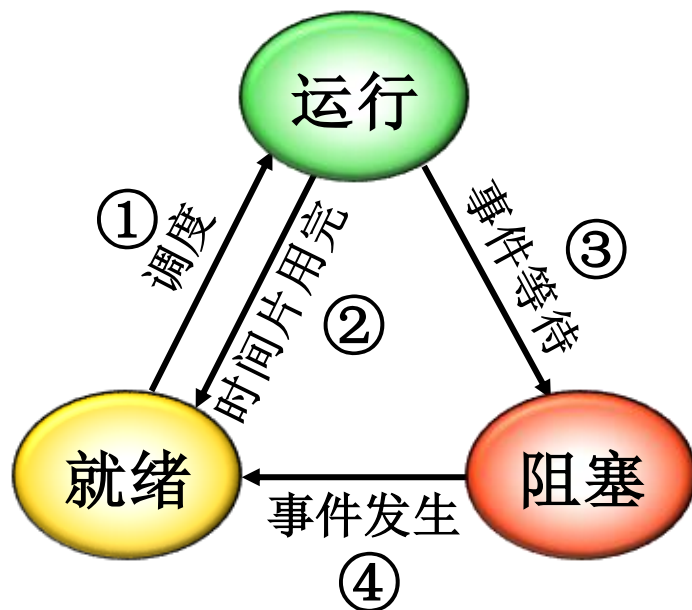
Q: 从就绪到等待可以吗?



3.2 进程的描述

3.2.4 进程的状态及其转换——三种基本状态及其转换

● 进程状态转换的相互影响



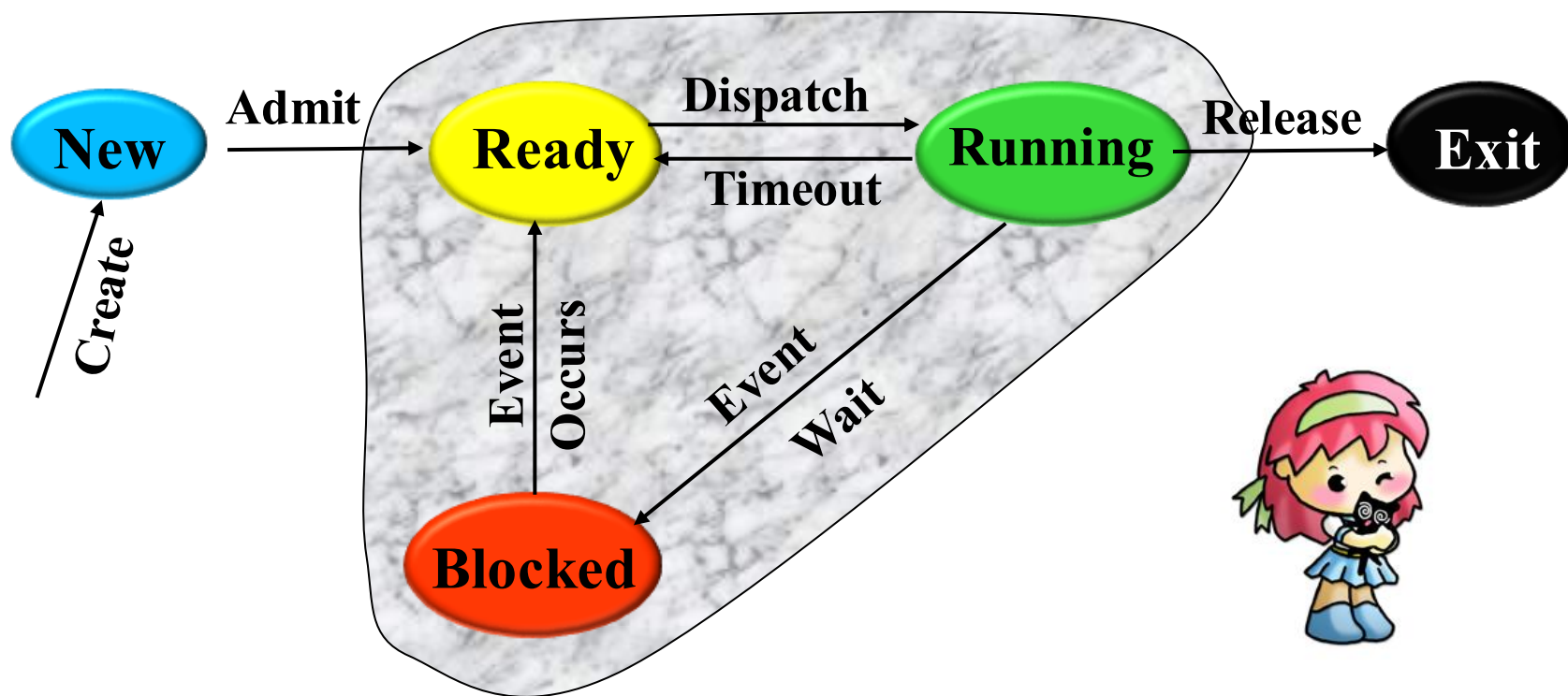
- ②必然引起哪个动作？①
- 如果存在就绪进程，③将引起哪个动作？①
- CPU空闲且无就绪进程时，④将引起哪个动作？①



3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

- 增加创建状态(**new**)、退出状态(**Exit**)





3.2 进程的描述



3.2.4 进程的状态及其转换-五状态进程模型

● 创建状态(new)

➤ OS已完成成为创建一进程所必要的工作

- ✓ 已构造了进程标识符;
- ✓ 已创建了管理进程所需的表格或数据结构。

➤ 因为资源有限，还没有允许执行该进程

- ✓ OS所需的关于该进程的信息保存在主存中的进程表中，但进程自身还未进入主存，也没有为与这个程序相关的数据分配空间，程序保留在辅存中；

➤ 导致进程创建的场景：

- ✓ 在批处理系统中，提交新作业
- ✓ 为新登录用户创建进程
- ✓ 请求打印的进程创建打印进程
- ✓ 由现有进程生成，父进程---子进程，等



3.2 进程的描述



3.2.4 进程的状态及其转换-五状态进程模型

● 终止状态 (Exit)

- 进程运行终止后，进程移入该状态
- 它不再有执行资格
- 资源已经释放，管理用表格和其它信息暂时保留
- 实用程序为了分析性能和利用率，可能要提取程序的历史信息

● 导致终止进程的原因

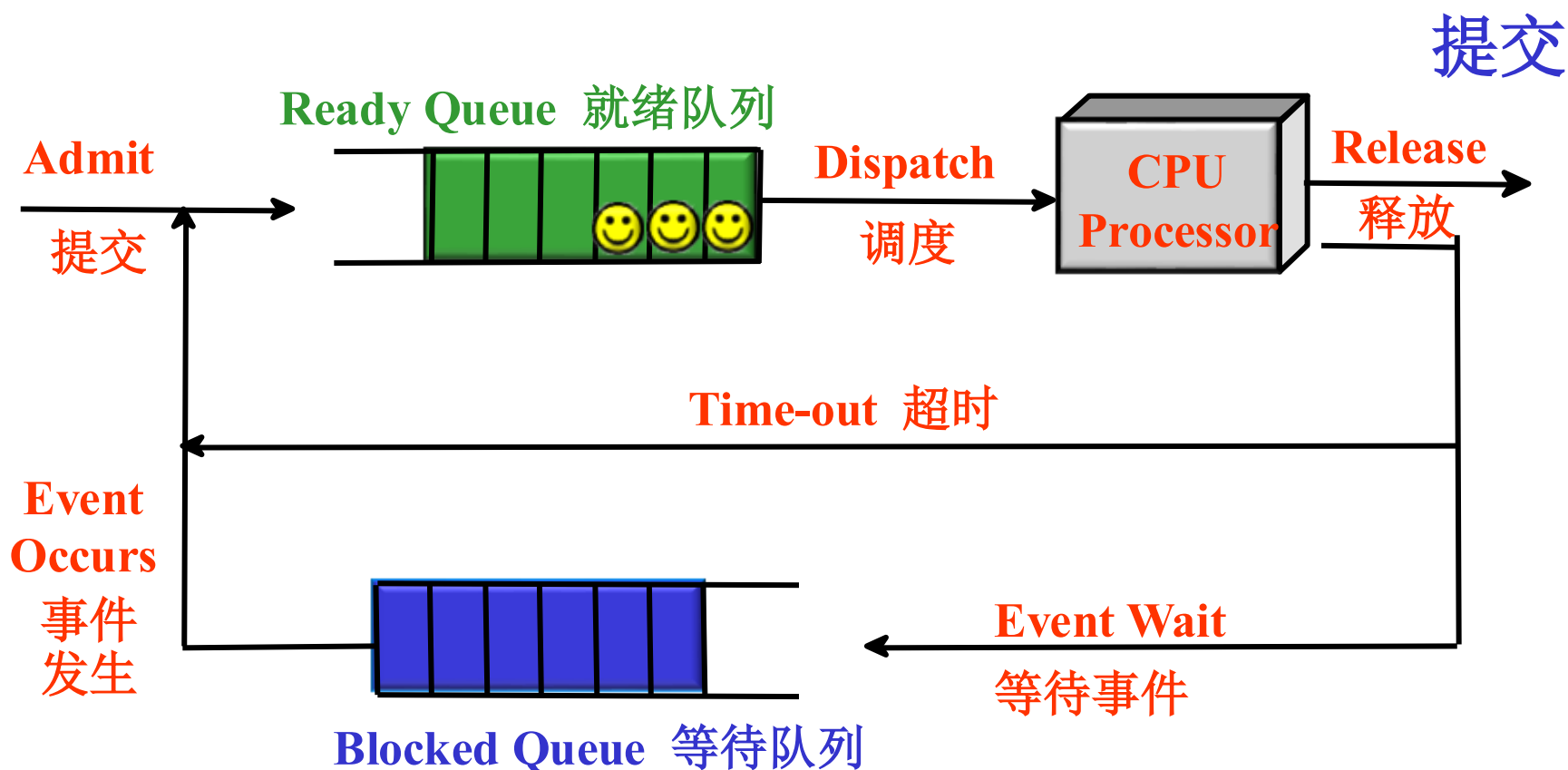
- ① 含**HALT指令**或用于终止的OS显式服务调用, 如exit();
- ② 分时系统中, **用户**的行为可指示终止, 如: 用户退出系统或关闭自己的终端, 该用户的进程将被终止
- ③ PC机环境中, **用户结束**应用程序
- ④ **出现某些错误**时, 如, I/O失败, 无效指令等
- ⑤ **父进程**可请求它的某个子进程终止
- ⑥ **父进程终止**, OS自动终止所有后代进程



3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 单队列结构

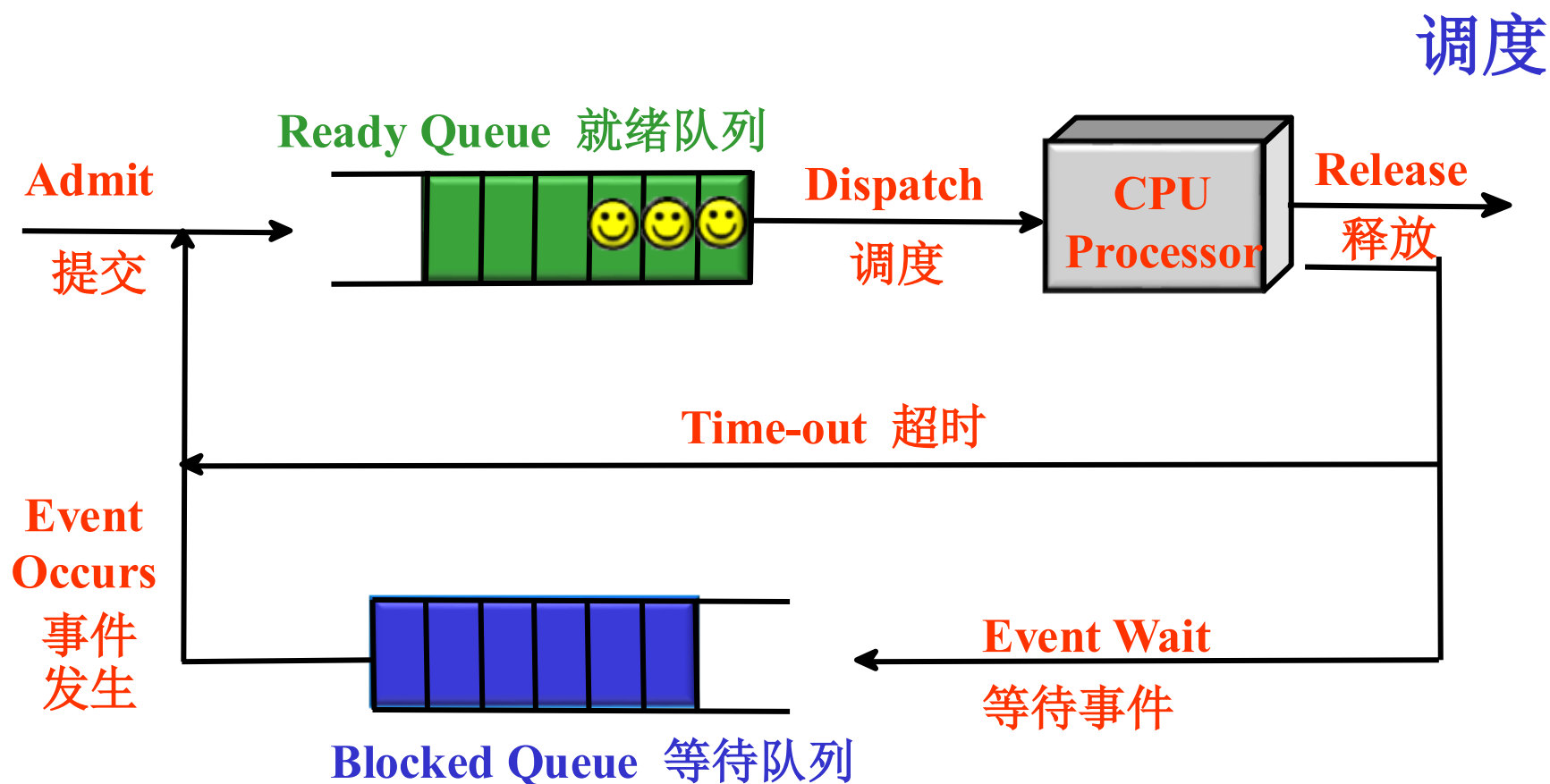




3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 单队列结构

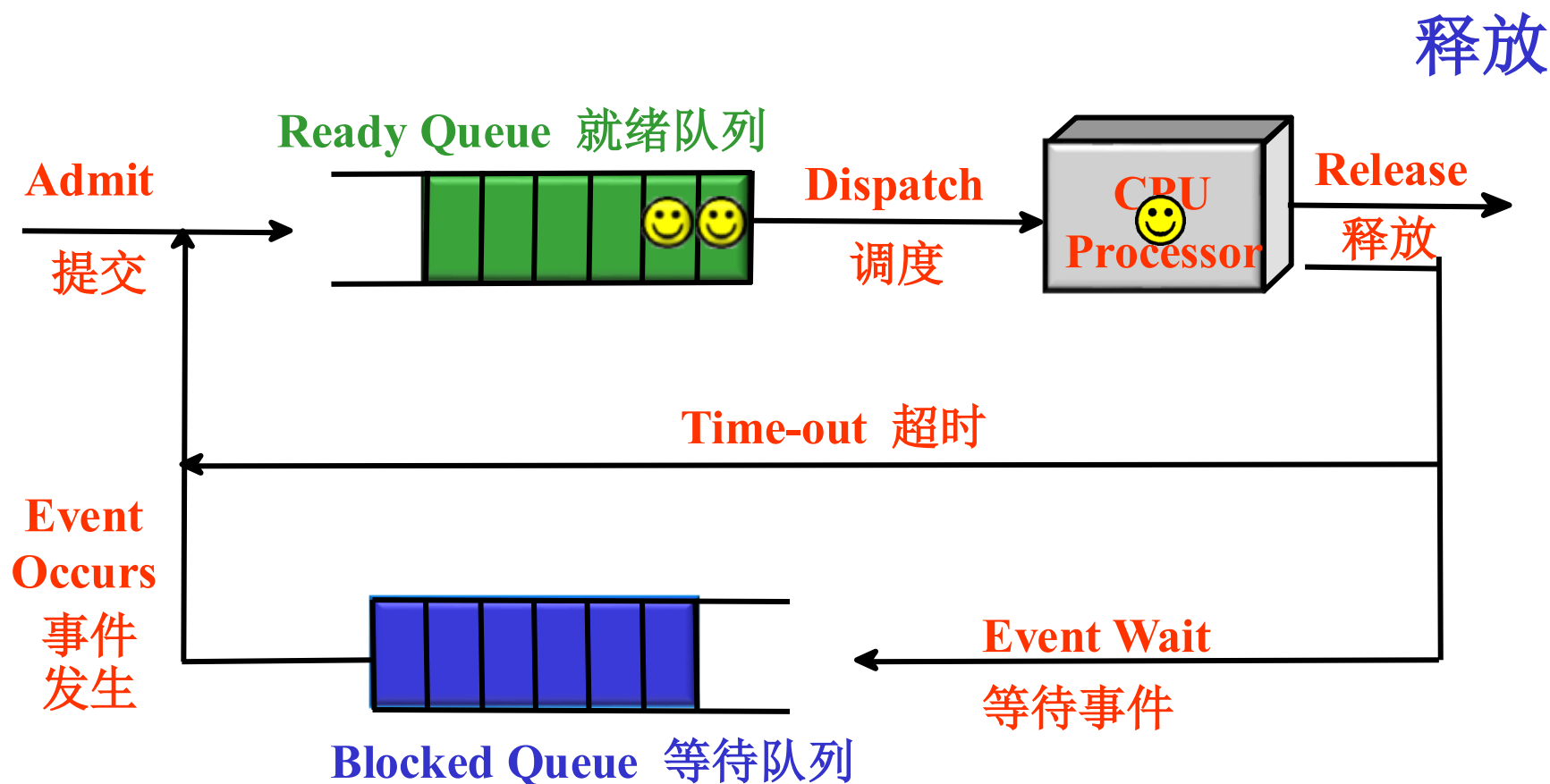




3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 单队列结构

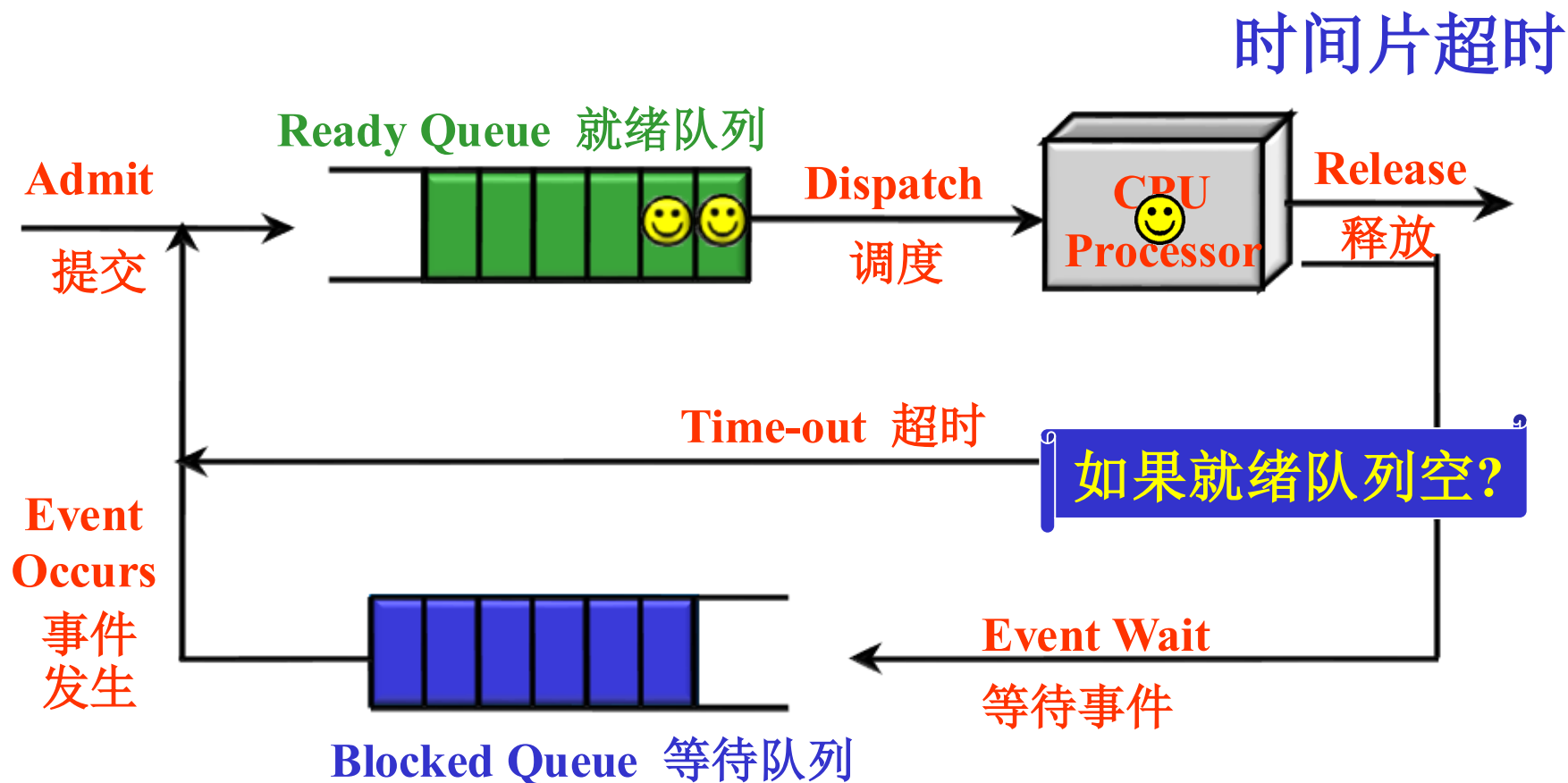




3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 单队列结构

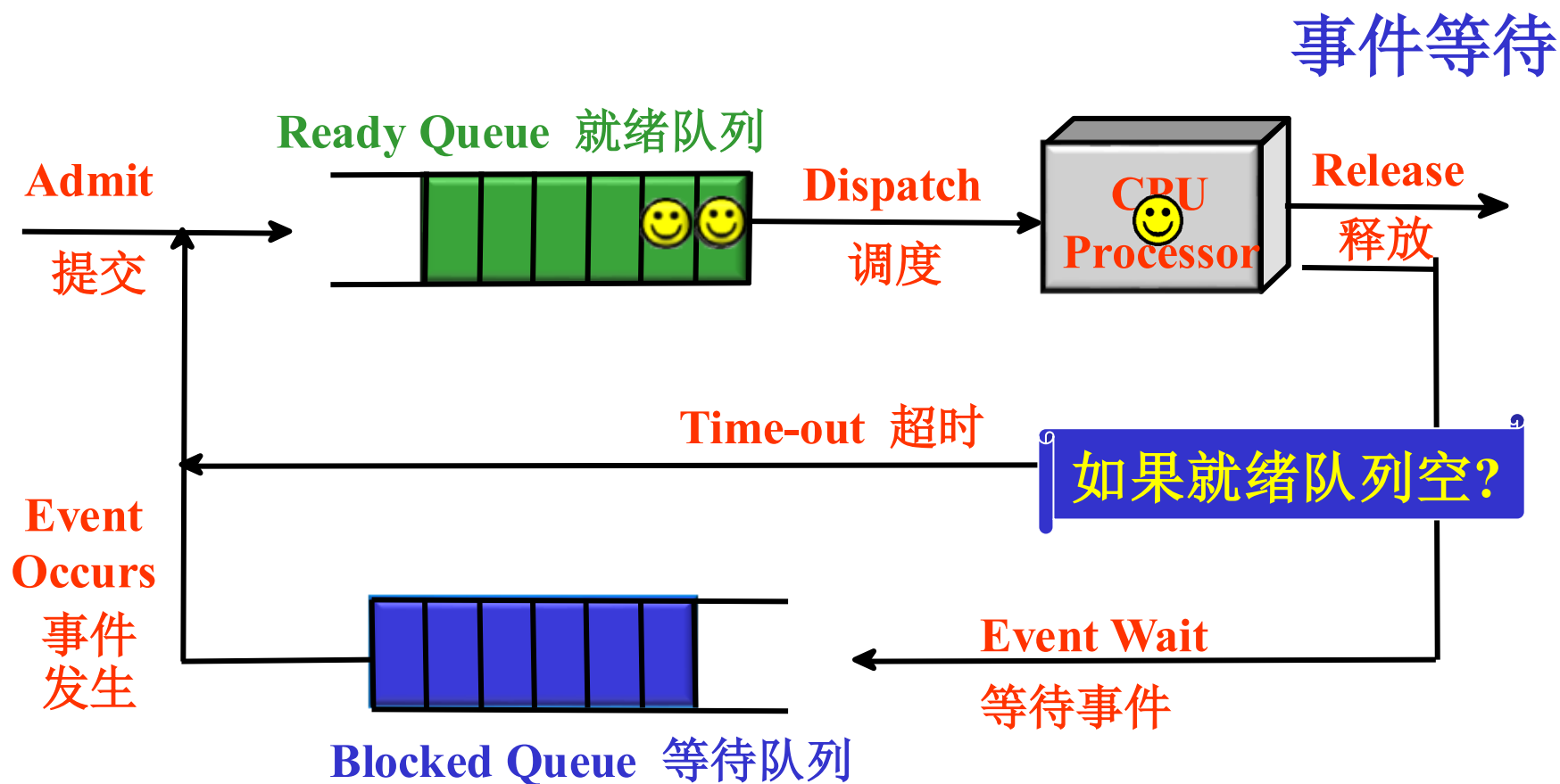




3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 单队列结构



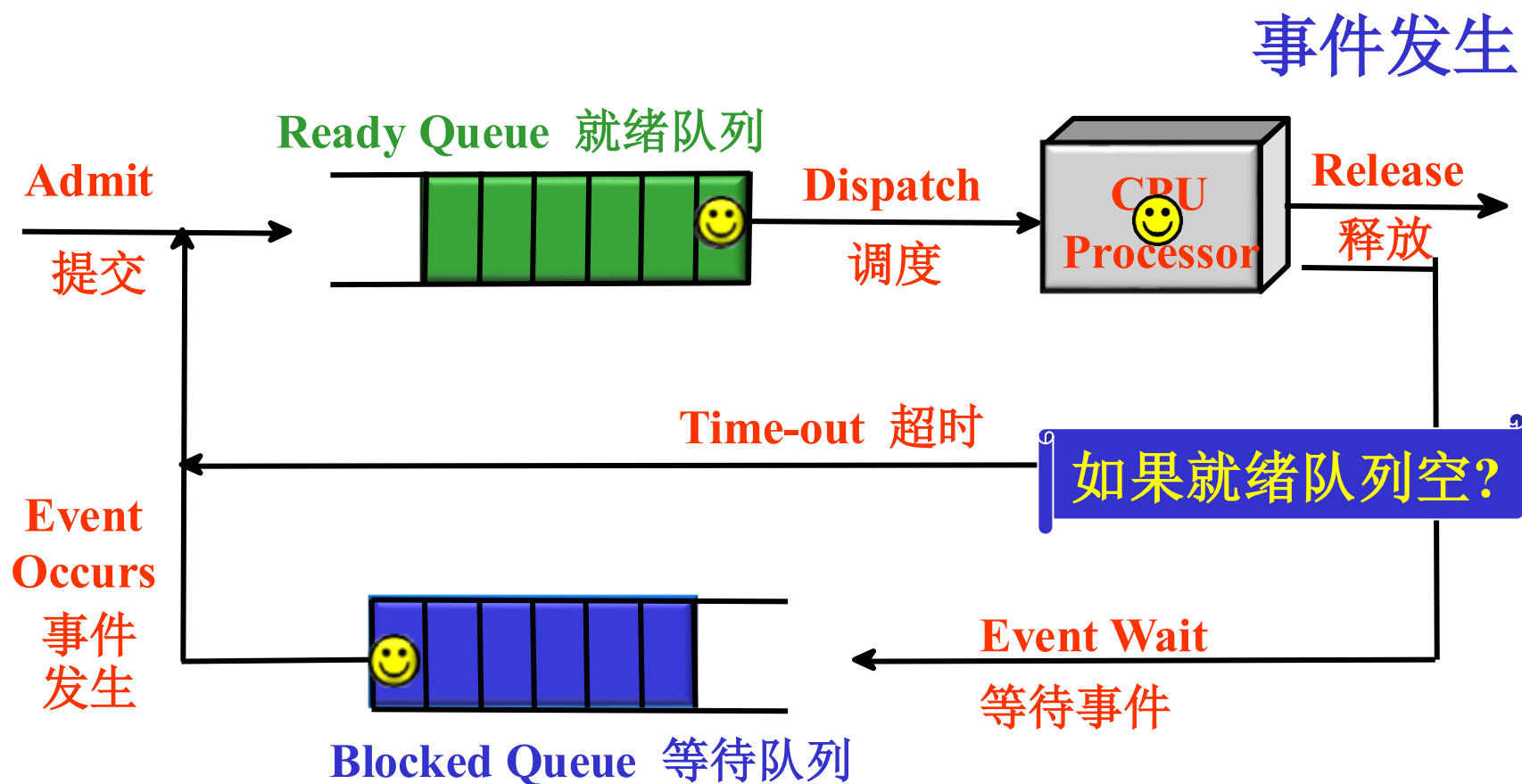


3.2 进程的描述



3.2.4 进程的状态及其转换-五状态进程模型

单队列结构

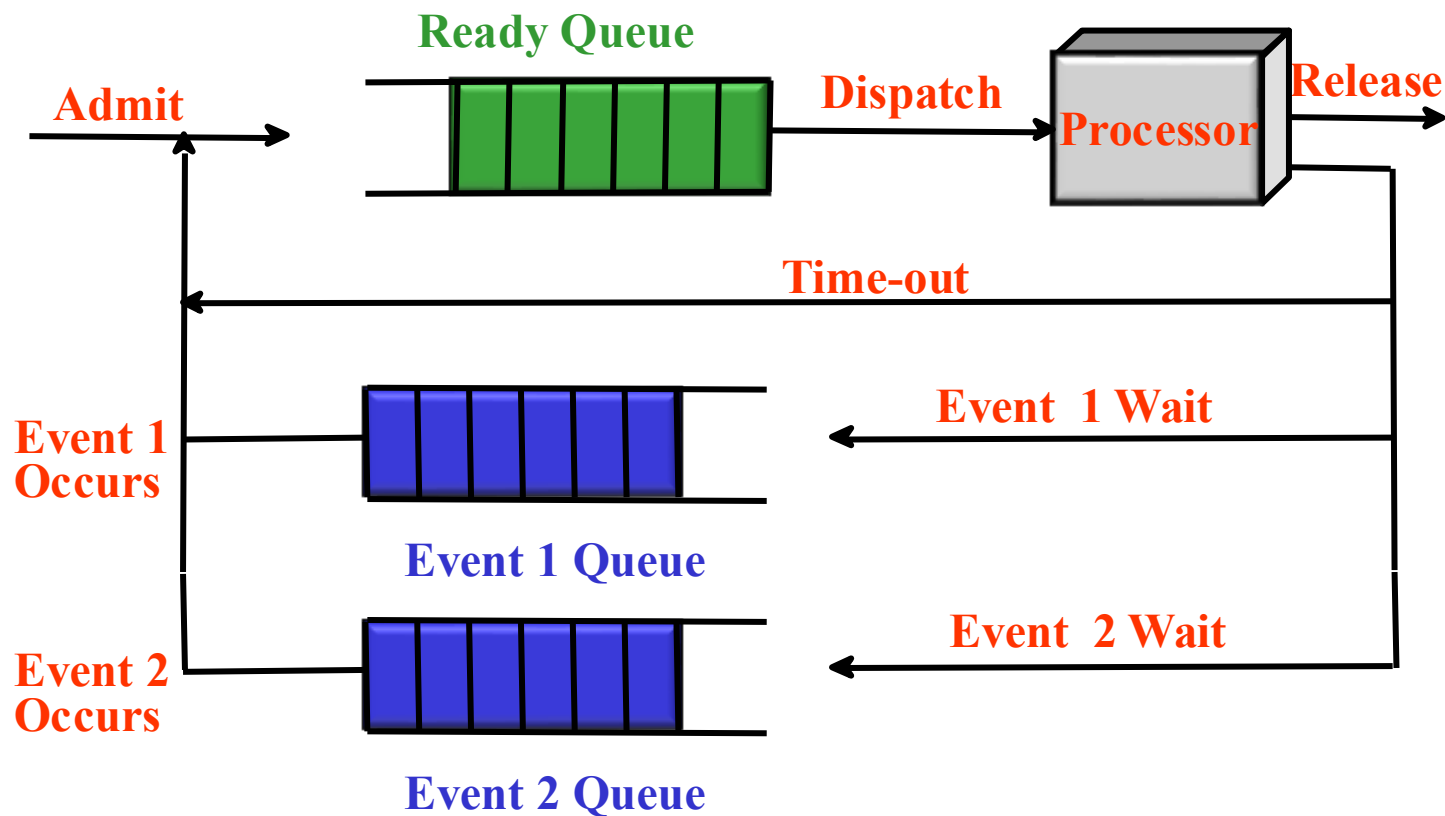




3.2 进程的描述

3.2.4 进程的状态及其转换-五状态进程模型

● 多队列结构





3.2 进程的描述



3.2.4 进程的状态及其转换-单挂起进程模型

- ✓ 原因：由于进程优先级的引入，一些低优先级进程可能等待较长时间，从而被对换至外存。
- ✓ 目的是：
 - 提高处理机效率：就绪进程表为空时，OS将阻塞进程从内存中“驱逐”到磁盘的“挂起队列”，再从该队列选另一进程进入内存，或接受一个新进程的请求。
 - 为运行进程提供足够内存：资源紧张时，暂停某些进程，如：CPU繁忙（或实时任务执行），内存紧张
 - 用于调试：在调试时，挂起被调试进程（从而对其地址空间进行读写）

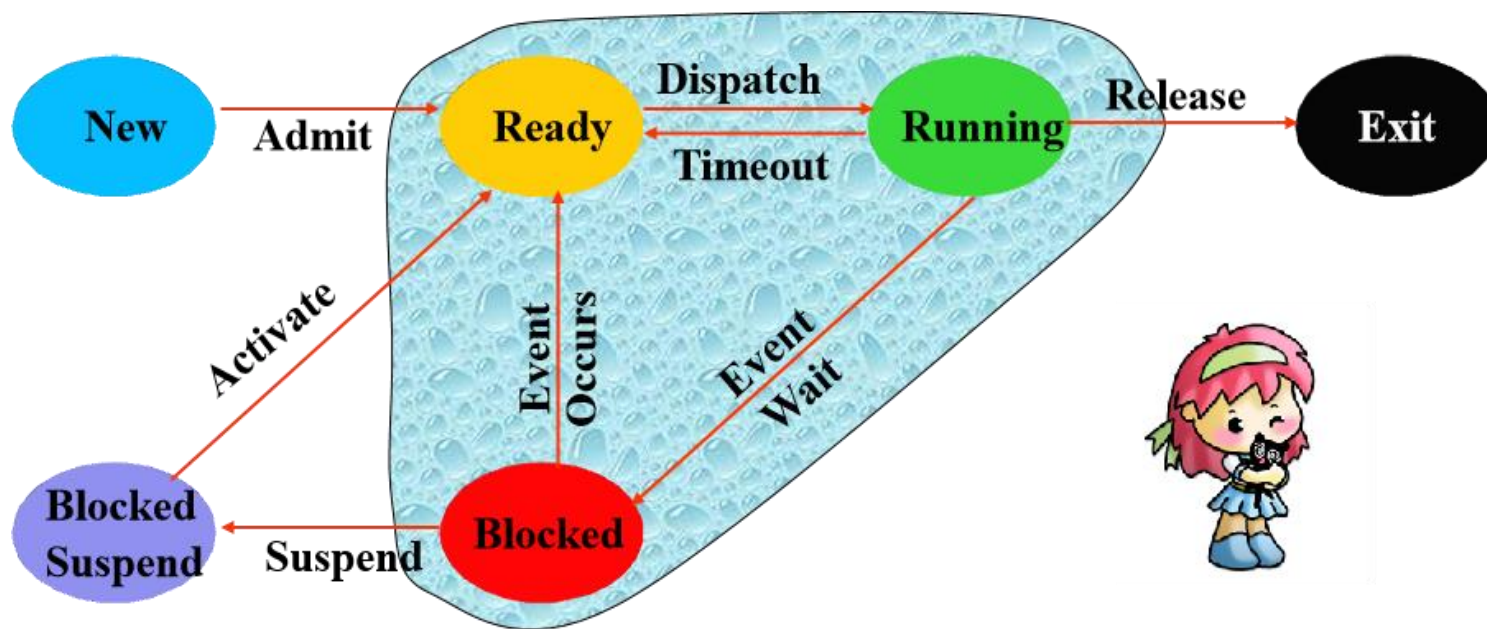


3.2 进程的描述



3.2.4 进程的状态及其转换-单挂起进程模型

- 做法：增加挂起状态
- 挂起状态：把一个进程从内存转到外存，可能是阻塞状态或就绪状态，目的是腾出内存空间



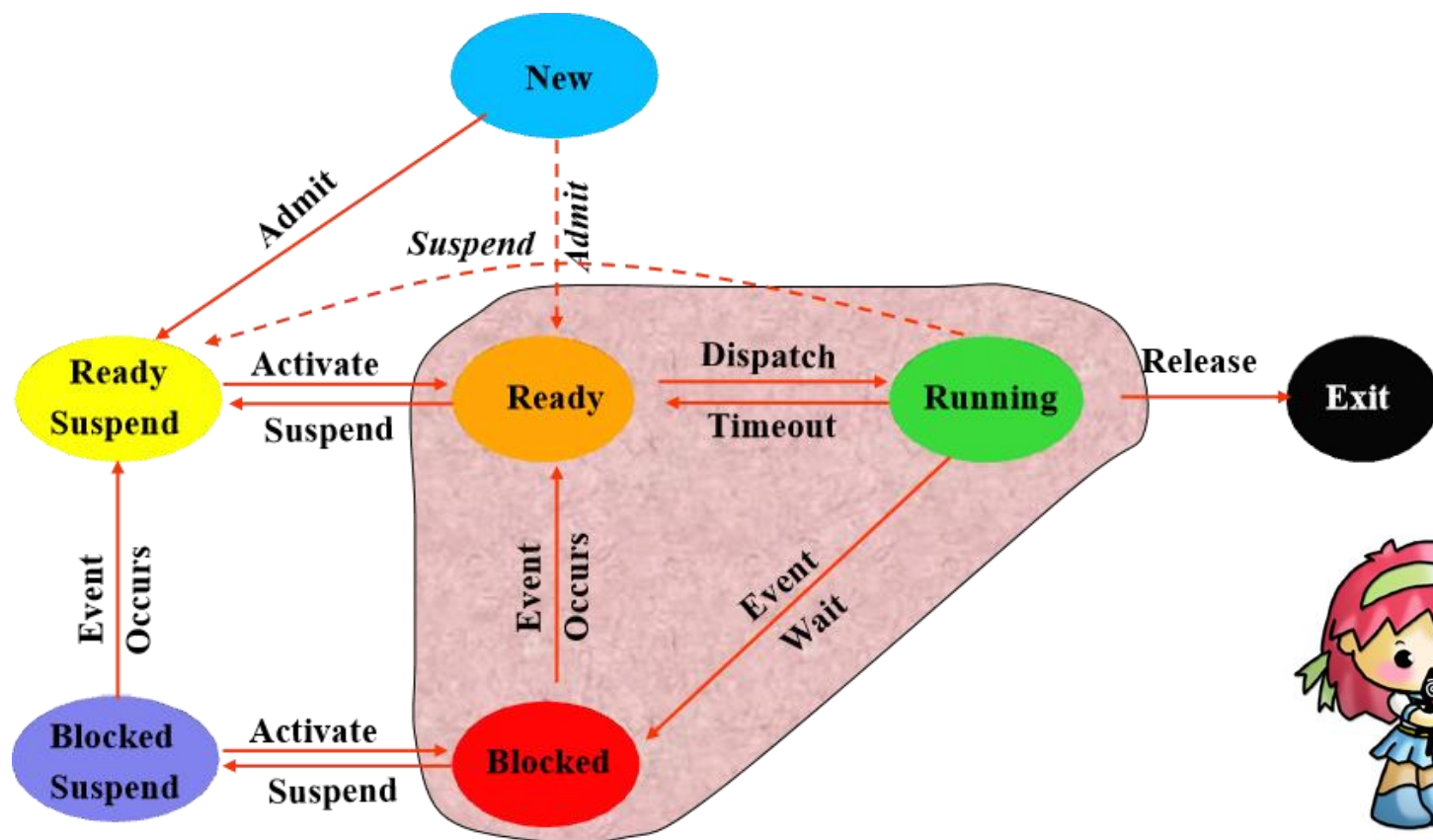


3.2 进程的描述



3.2.4 进程的状态及其转换-双挂起进程模型

- **做法：** 增加就绪挂起状态、阻塞挂起状态





3.2 进程的描述



3.2.4 进程的状态及其转换-双挂起进程模型

挂起进程模型的状态需考虑二方面：**是否就绪，是否在内存**

- **就绪状态(Ready)**: 进程在内存且可立即进入运行状态;
- **阻塞状态(Blocked)**: 进程在内存, 并等待某事件的出现;
- **就绪挂起状态 (Ready, suspend)**: 进程在外存, 但只要进入内存, 即可运行;
- **阻塞挂起状态 (Blocked, suspend)**: 进程在外存并等待某事件的出现。



3.2 进程的描述



3.2.4 进程的状态及其转换-双挂起进程模型

● 状态间的转换

- **挂起（Suspend）**：把一个进程从内存转到外存；可能有以下几种情况：
 - ✓ **阻塞到阻塞挂起**：没有进程处于**就绪状态**或就绪进程要求更多内存资源时，会进行这种转换，以纳入新进程或运行就绪进程；
 - ✓ **就绪到就绪挂起**：当有**高优先级阻塞**（系统认为会很快就绪的）进程和低优先级就绪进程时，系统会选择挂起低优先级就绪进程；
 - ✓ **运行到就绪挂起**：对**抢先式**分时系统，当有**高优先级阻塞**挂起进程因事件出现而进入就绪挂起时，系统可能会把**运行的进程**转到**就绪挂起状态**。



3.2 进程的描述

3.2.4 进程的状态及其转换-双挂起进程模型

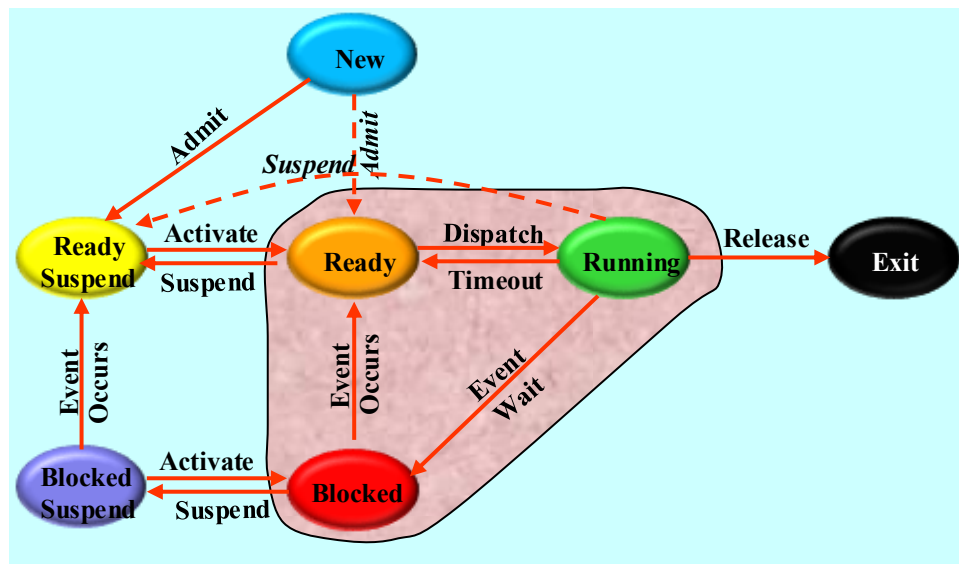
● 状态间的转换

➤ **激活 (Activate)**：把一个进程从外存转到内存；可能有以下几种情况：

✓ **就绪挂起到就绪**：没有就绪进程或挂起就绪进程优先级高于就绪进程时，会进行这种转换；

✓ **阻塞挂起到阻塞**：

当一个进程**释放足够内存**时，系统会把一个高优先级阻塞挂起（系统认为会很快出现所等待的事件）进程转为阻塞状态；较少出现。





3.2 进程的描述



3.2.4 进程的状态及其转换-双挂起进程模型

● 状态间的转换

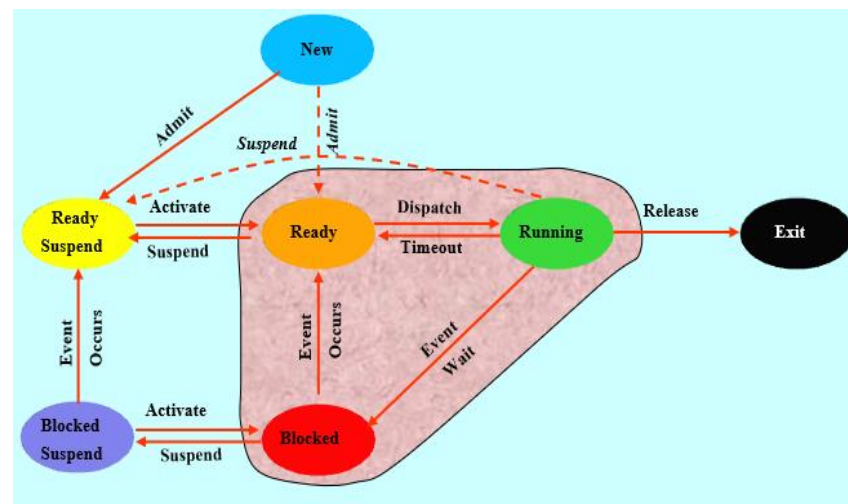
➤ 事件出现 (Event Occurs)：进程等待的事件出现；
如：操作完成、申请成功等；可能的情况有：

● 阻塞到就绪：针对内存进程的事件出现；

● 阻塞挂起到就绪挂起：针对外存进程的事件出现；

➤ 收容(Admit)：收容一个新进程，进入就绪状态或就绪挂起状态。

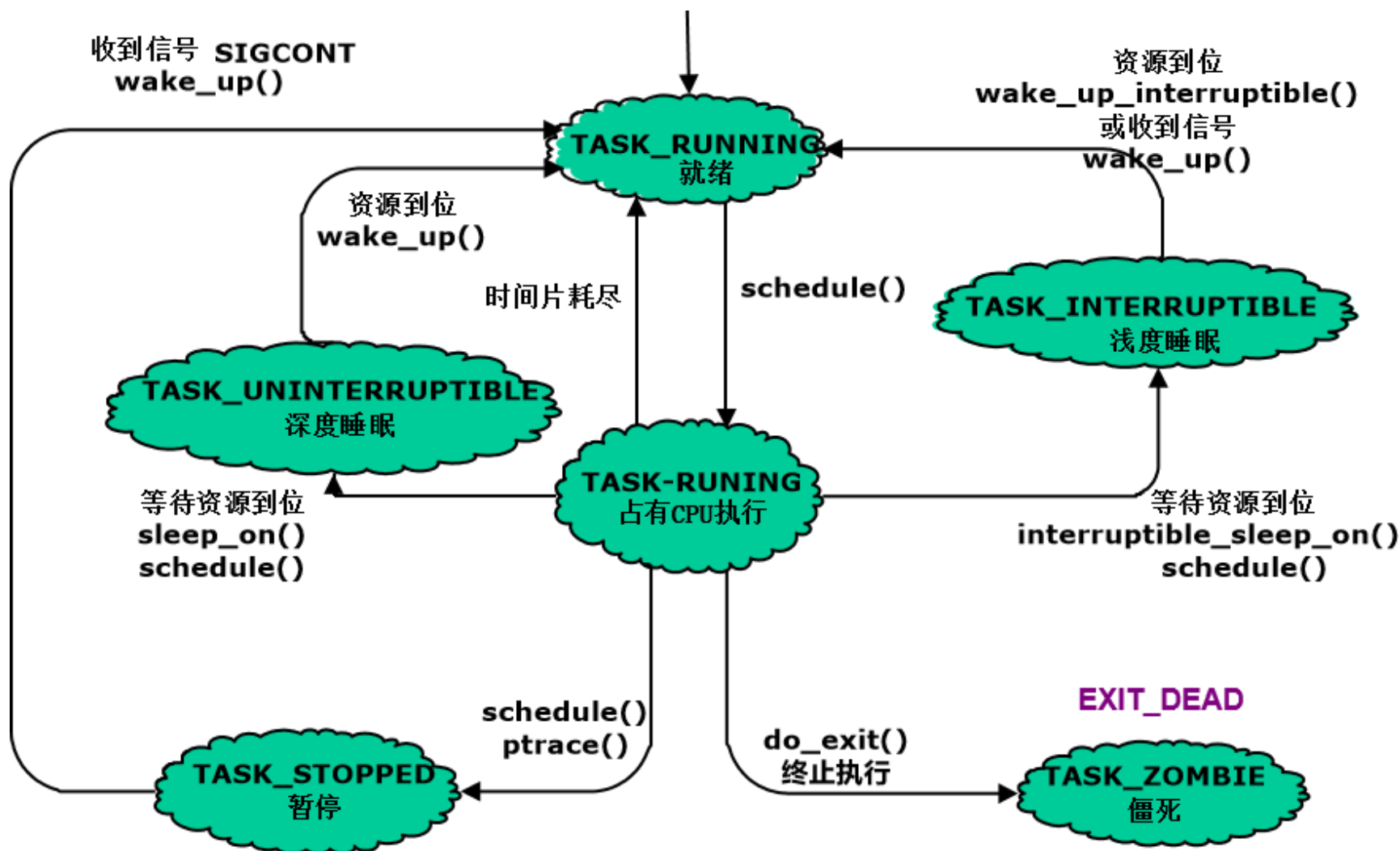
➤ 各种状态---退出：被父进程终止或父进程本身终止。





3.2 进程的描述

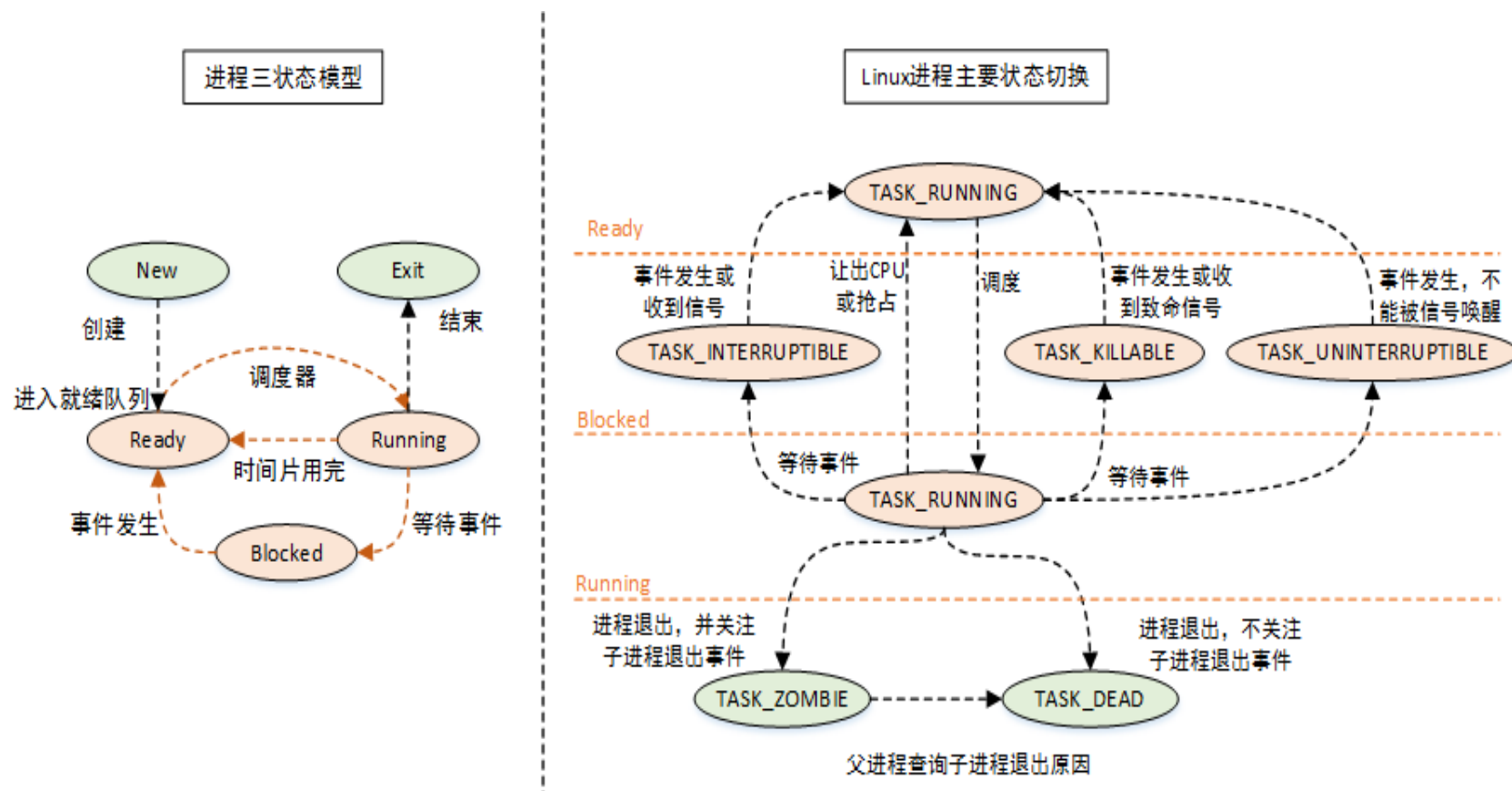
3.2.5 Linux(openEuler)进程状态转换图





3.2.5 Linux(openEuler)进程状态转换图

- 高版本内核中增加了TASK_KILLABLE状态



源自: (一) Linux进程调度器-基础, <https://www.cnblogs.com/LoyenWang/p/12249106.html>



3.2 进程的描述

3.2.6 PCB的组织方式

- **问题：** 如何组织系统中众多的PCB？
- **做法：** PCB表、链表、索引表、...





3.2 进程的描述

3.2.6 PCB的组织方式

(1) PCB表

- 系统把所有PCB组织在起来，并把它们放在内存的**固定区域**，就构成了**PCB表**；
- PCB表的大小决定了系统中最多可同时存在的进程个数，称为系统的**并发度**。
- 特点：
 - 空间预留，造成浪费
 - 最大进程数目受限

PCB1
PCB2
PCB3
PCB4
PCB5
PCB6
PCB7
PCB8
PCB9
..... https://blc

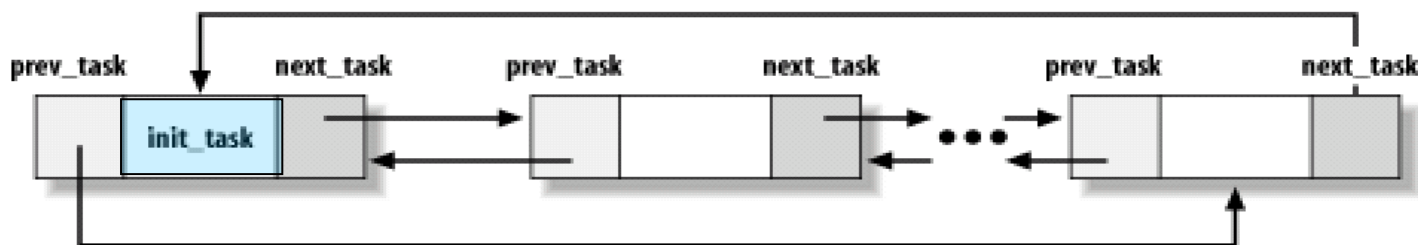
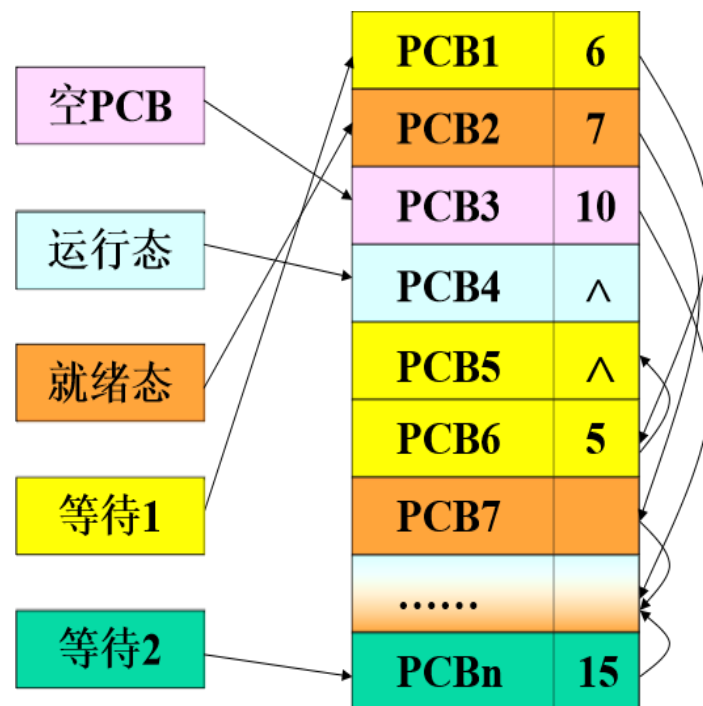


3.2 进程的描述

3.2.6 PCB的组织方式

(2) 链表（队列）

- ✓同一状态的进程其PCB成一链表，多个状态对应多个不同的链表；
- ✓例如：就绪链表、等待链表。



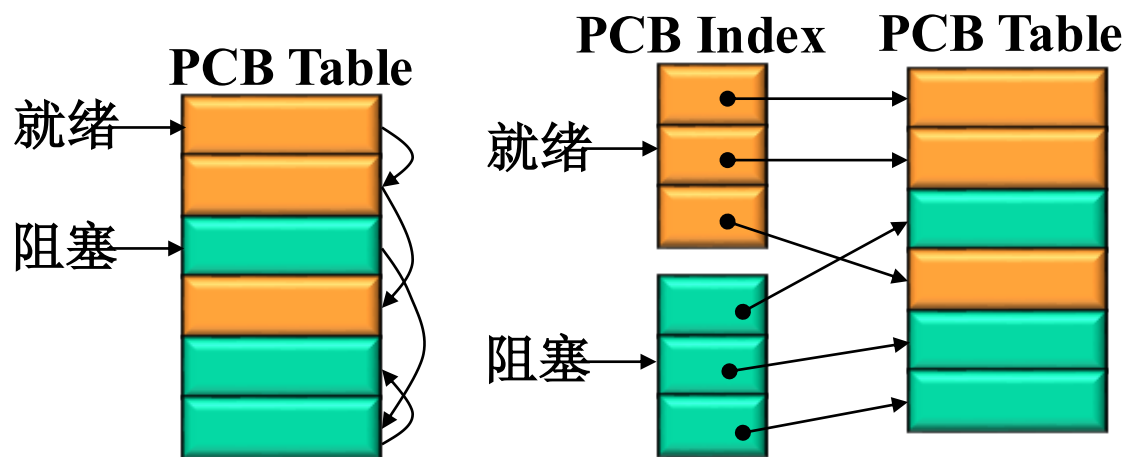


3.2 进程的描述

3.2.6 PCB的组织方式

(3) 索引表

- 同一状态的进程归入一个index表（由index指向PCB），多个状态对应多个不同的index表；
- 各状态的进程形成不同的索引表：就绪索引表、阻塞索引表。
- **问题：**索引表如何构造？大小可变或者某类进程的数量是可变的





3.2 进程的描述



- 小结

- 进程组成
- 进程控制块
- 进程上下文
- 进程控制块的组织
- CPU的两种运行状态
- 进程状态与转换：
 - 3基本状态
 - 2+3状态
 - 5+1挂起状态（suspend）
 - 5+2挂起状态（suspend）
- Linux(openEuler)进程的6、7种状态



3.2 进程描述-思考



练习思考题

1. 如果系统中有N个进程，运行的进程最多几个，最少几个；就绪进程最多几个最少几个；等待进程最多几个，最少几个？
2. 有没有这样的状态转换，为什么？
等待—运行； 就绪—等待
3. 一个状态转换的发生，是否一定导致另一个转换发生，列出所有的可能。
4. 举3个日常生活中类似进程的例子
5. 阅读博文“Linux进程描述符task_struct结构体详解”
(https://www.cnblogs.com/x_wukong/p/8496646.html)，了解Linux4.5内核的进程描述符（即进程控制块）结构，试回答Linux的PCB是如何组织和存放的？

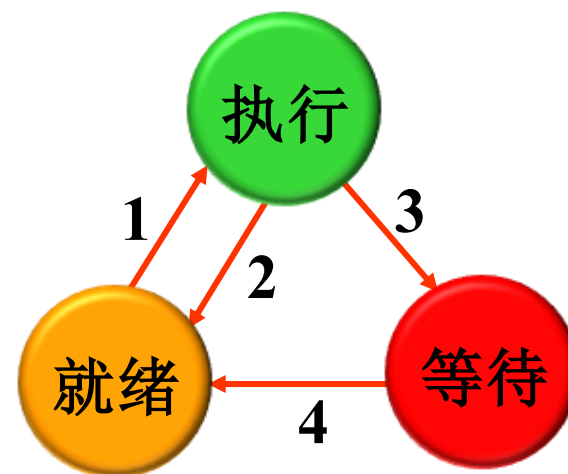


3.2 进程描述-思考

● 自主设计题

- 进程的3个基本状态：执行、就绪、等待，其转换标识分别为1、2、3、4（如图）。编程模拟进程状态转换过程，要求：分别建立等待队列和就绪队列（单队列、结构不限），初始状态时，1个进程处于执行状态，7个进程在等待队列中，7个进程在就绪队列中。手工触发任意一种转换，显示转换发生后的执行进程和两个队列的进程。

（提示：参考五状态进程模型（单队列结构）的动画，编程实现该动画的效果）





Review 6



1. 进程概述

- 进程定义、特征、组成
- 进程与程序的区别

2. 进程描述

- PCB: 作用、内容、组织 (PCB表、链表、索引)
- 进程上下文
- 进程状态: 3、5、6、7
- 状态间的转换

3. CPU的运行状态: 用户态、内核态

4. Linux(openEuler)进程的6、7种状态



内容提要



几个问题：

1. 如何创建进程？创建都干些什么？
2. 进程状态转换如何实现？
3. 进程如何撤销？撤销都做些什么？
4. UNIX如何创建进程？
5. AI时代**教与学的问题**：教师如何教，学生如何学？建议想法



3.3 进程控制



为了描述程序在并发执行时对系统资源的共享，需要一个描述程序执行时动态特征的概念，这就是进程。本章将讨论进程概念、进程控制和进程间关系。

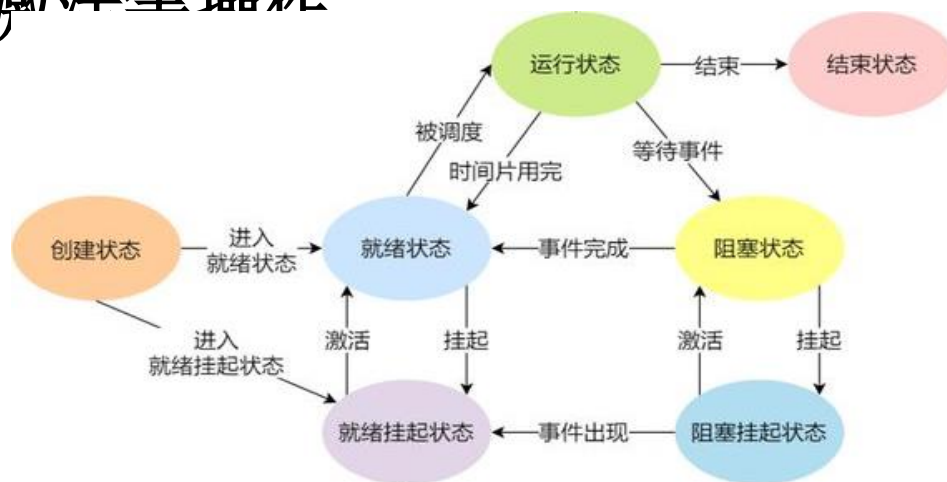
1. 概述
2. 进程的描述
3. 进程控制
4. 线程
5. 进程互斥和同步
6. 进程间通信
7. 死锁问题
8. 进程其他方面的举例



3.3 进程控制

3.3.1 进程控制的概念

- **进程控制**：指系统使用一些具有特定功能的程序段来**创建、撤销**进程以及完成进程各**状态间的转换**，从而达到进程**高效率**并发执行和协调、实现资源共享的目的。
- **进程控制主要完成**：进程创建、撤销、阻塞、唤醒、挂起、~~激活~~操作





3.3 进程控制

3.3.1 进程控制概念

- **原语(primitive):** 在系统状态下执行的具有特定功能的，作为一个整体而不可分割执行的程序段称为原语。即原语是要么全都完成，要么全都不做。许多系统调用就是原语。
- 原语分为两类：
 - 机器指令级：不允许中断
 - 功能级原语：允许中断，不允许并发
- **注意：**系统调用并不都是原语，例如read()。
 - 进程A调用read()，因无数据而阻塞，在read()里未返回。然后进程B调用read()，此时read()被重入。系统调用不一定一次执行完并返回该进程，有可能在特定的点暂停，而转入到其他进程。

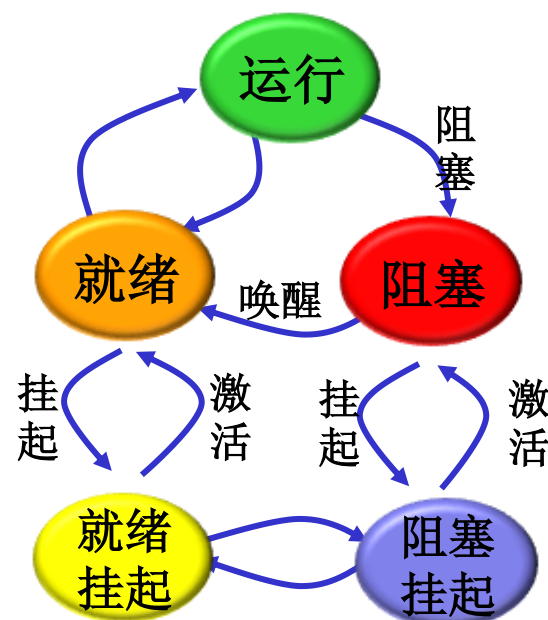


3.3 进程控制



3.3.2 进程控制的主要原语

- 完成进程创建、撤销，以及进程的状态转换都需要执行一段程序来完成相应的功能。
- 它们若不是原语呢？
- 目的：**按原语对待可以降低系统的开销和复杂度
- 进程控制原语包括：
 - 进程创建与撤消原语
 - 进程阻塞与唤醒原语
 - 进程挂起与激活原语





3.3 进程控制

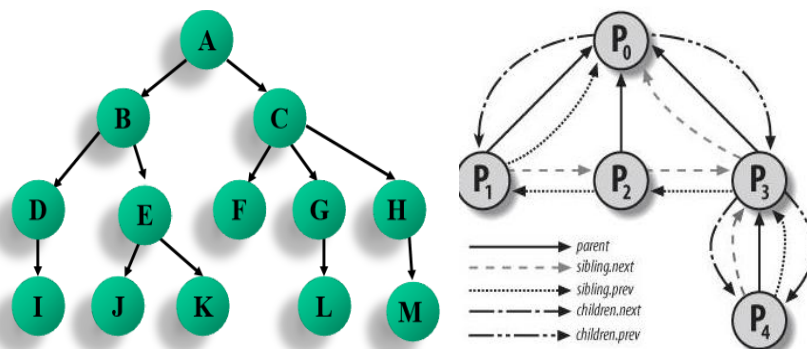
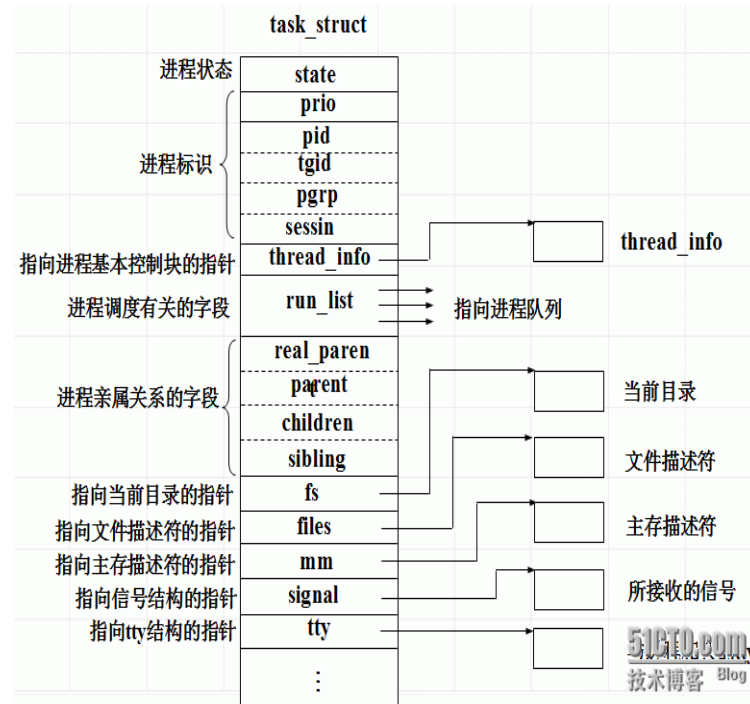
3.3.2 进程控制的主要原语

(1) 进程创建原语

应该干些什么？

主要任务：

- ① 为进程创建一个PCB
- ② 赋予一个唯一进程标识符
- ③ 为进程映像分配空间
- ④ 初始化PCB
- ⑤ 设置相应的链接：
 - 把新进程加到就绪队列中
 - 插入家族队列——进程树





3.3 进程控制



3.3.2 进程控制的主要原语

(1) 进程创建原语

● 子进程的创建的2种形式

- **fork+exec**: 复制现有进程的上下文, 但是新进程的系统上下文会有不同, 例如PCB、栈不同。要执行新的程序, 需要exec()的配合。
- **spawn ([spɔ:n])**: 创建新进程并加载新程序, 直接产生执行程序的子进程。Windows采用这种机制。



3.3 进程控制

3.3.2 进程控制的主要原语

(1) 进程创建原语

- **继承(inherit)的信息：**子进程可以从父进程中继承**用户标识符、环境变量、打开文件、文件系统的当前目录、控制终端、已经连接的共享存储区、信号处理例程入口表等。**
- **不被继承信息：**进程标识符、父进程标识符。



3.3 进程控制



3.3.2 进程控制的主要原语

(1) 进程创建原语

Procedure Create(S0, P0, M0, R0)

Begin

i := getEmptyPCB(); //查找PCB集合, 返回一个可用PCB的内部标识符 i

i.id := getPid(); //被创建进程的外部标识符

i.priority := P0; /* 初始化PCB, 进程优先级 */

i.CPU_State := S0; //处理机的初始状态

i.Mainstore := M0; //初始内存数

i.resources := R0; //所需资源清单

i.status := Ready;

j := EP; i.parent := j; /* 插入家族队列 */

i.progeny := Λ ; j.progeny := i;

i.sdata := RQ; //进程状态有关的指针

insert(RQ, i); /* 插入就绪队列 */

End;



3.3 进程控制



3.3.2 进程控制的主要原语

(2) 进程撤销原语Destroy

● 导致进程被撤消的原因

- ① 一个进程在完成任务后正常终止
- ② 由于错误导致非正常终止
- ③ 祖先进程要求撤消某个子进程

● 处理过程:[做什么?]

- ① 释放内外存空间
- ② 关闭所有打开文件或者是共享计数减一
- ③ 释放共享内存段和各种锁定lock
- ④ 撤消其子孙进程（以免其成为不可控的）
- ⑤ 释放其PCB



3.3 进程控制

3.3.2 进程控制的主要原语

(2) 进程撤销原语Destroy

● 撤消原语的描述

调用者提供的进程的外部标识

Procedure Destroy(n)

Begin

Sched := false;

kill(n);

**if Sched then Scheduler()
else continue;**

End;

Procedure kill(n)

Begin

i := GetInternalName(n);

if i.status = "Running" then

stop(i); Sched := True;

remove(i.sdata, i);

for all S ∈ i.progeny do kill(S);

for all r ∈ MainstoreUi.resources do

Release(r);

Release (PCB(i))

End; (Kill是可递归的)

如果进程正在运行，则立即停止其运行，并设置调度标志为真



3.3 进程控制



3.3.2 进程控制的主要原语

(3) 进程阻塞原语Block

● 阻塞原因:

- 当进程期待的某事件尚未出现时，该进程调用阻塞原语把自己阻塞起来。

● 处理过程:

- ① 保护中断现场
- ② 置进程的状态为“等待态”
- ③ 将其插入该事件的等待队列WQ(r)
- ④ 转进程调度

- 注意: 由自己阻塞自己，但该进程的唤醒则是在该进程所期待的事件发生后，由“发现者”进程调用唤醒原语实现。



3.3 进程控制



3.3.2 进程控制的主要原语

(3) 进程阻塞原语Block

阻塞进程的描述

*/*r 为指向等待原因的对应等待队列的指针*/*

Procedure block(r)

Begin

i := EP; *//从执行进程的指针获得进程的內部标识符i*

stop(i); *//停止进程i*

i.status := “blocked”;

i.sdata := WQ(r);

insert(WQ(r), i);

scheduler(); *//进程调度程序*

End;



3.3 进程控制



3.3.2 进程控制的主要原语

(4) 进程唤醒原语Wakeup

● 唤醒原因:

➤ 进程等待的事件发生，等待队列中的进程被唤醒。

● 唤醒进程的两种方法:

- ① 由系统进程唤醒。系统进程统一控制事件的发生，并将“事件发生”这一消息通知等待进程。等待的是公共资源；
- ② 由事件发生进程唤醒。事件发生进程与被唤醒的进程是合作关系，等待私有资源。



3.3 进程控制



3.3.2 进程控制的主要原语

(4) 进程唤醒原语Wakeup)

唤醒原语的描述

Procedure wakeup(n)

Begin

i := GetInternalName(n);

remove(WQ(r), i); //从相应的等待队列中摘下被唤醒的进程

i.status := “Ready”; //状态置为 “就绪态ready”

i.sdata := RQ;

insert(RQ, i); //插入就绪队列

continue; //唤醒进程返回或转进程调度

End;



3.3 进程控制

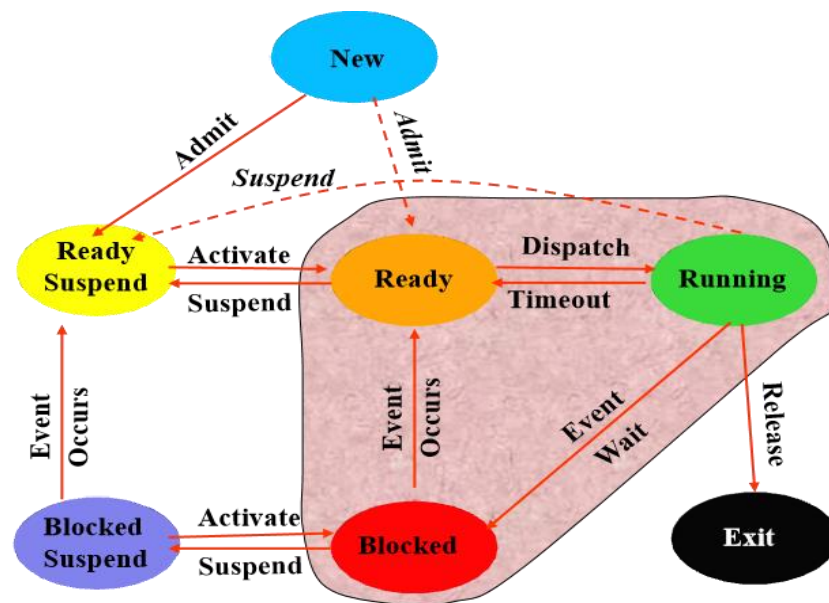
3.3.2 进程控制的主要原语

(5) 挂起原语 (Suspend Primitive)

● **需求：** 当需要把某进程置于就绪挂起状态 (Readys) 或阻塞挂起状态(Blockeds)时，调用挂起原语。

● **进程的挂起方式：**

- 发命令的进程**自身**挂起；
- 挂起具有**指定标识符**的进程；
- 将某进程及其全部或部分“子孙”进程挂起。





3.3 进程控制



3.3.2 进程控制的主要原语

(5) 挂起原语 Suspend Primitive

● 挂起原语的描述

```
procedure Suspend(n, a)      /* 挂起指定标识符n的  
Begin                          进程的进程原语*/  
    i := GetInternalName(n);  /* 参数a是用于保存该  
    S := i.status;           进程PCB副本的内存区*/  
    if S="Running" then stop(i);  
    a := copy PCB(i);  
    if S="blocked" then i.status := "blockeds"  
    else i.status := "readys"  
    if S="Running" then Scheduler() else continue;  
End;
```



3.3 进程控制

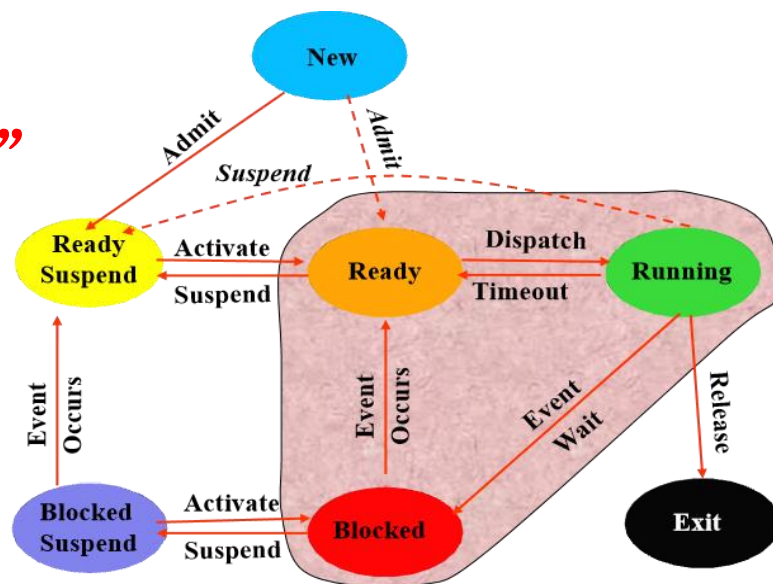
3.3.2 进程控制的主要原语

(6) 激活原语 (Activate Primitive)

● **功能：**使处于挂起状态的进程变为活动状态，即把“挂起就绪”变为“活动就绪”，把“挂起阻塞”变为“活动阻塞”。

● 激活方式

- ① 激活一个**指定标识符**的进程
 - ② 激活**某进程及其所有“子孙”**进程等
- 当激活的进程处于“活动就绪”状态时，可能引起进程的重新调度





3.3 进程控制

3.3.2 进程控制的主要原语

(6) 激活原语 (Activate Primitive)

● 激活原语的描述

```
procedure activate(n)      /*n为进程标志符*/  
begin  
    i := GetInternalName(n);  
    if (i.status= “readys” ) then  
        i.status :=“readya”  
    else i.status := “blockeda”;  
    if i.status = “readya” then scheduler();  
    else continue  
End;
```



3.3 进程控制



3.3.3 Unix的进程管理

- 系统上电—>引导程序自举—>OS进入内存—>OS初始化, 建立0#进程、1#进程—>为各终端创建用户交互进程(Shell)和终端管理进程—>出现提示符—>接收用户命令—>创建新用户进程 —>进程树, 1#进程是祖先。

● UNIX的进程上下文

● UNIX的进程状态及其转换

● UNIX的进程控制

- ✓ 创建与撤消
- ✓ 进程阻塞与唤醒、换入换出



3.3 进程控制



3.3.3 Unix的进程管理

(1) UNIX的进程上下文

● 用户级上下文

- 进程正文：程序中可执行的机器指令
- 进程数据：由这个进程访问的数据
- 用户栈：过程调用时的参数、局部变量、返回地址
- 共享存储区：与其他进程共享的存储区，用于进程间的通信



3.3 进程控制



3.3.3 Unix的进程管理

(1) UNIX的进程上下文【略讲】

●寄存器级上下文：

- 程序计数器
- 状态寄存器
- 栈指针（指向用户栈或核心栈的栈顶，取决于当前的运行模式）
- 通用寄存器



3.3 进程控制



3.3.3 Unix的进程管理

(1) UNIX的进程上下文

● 系统级上下文

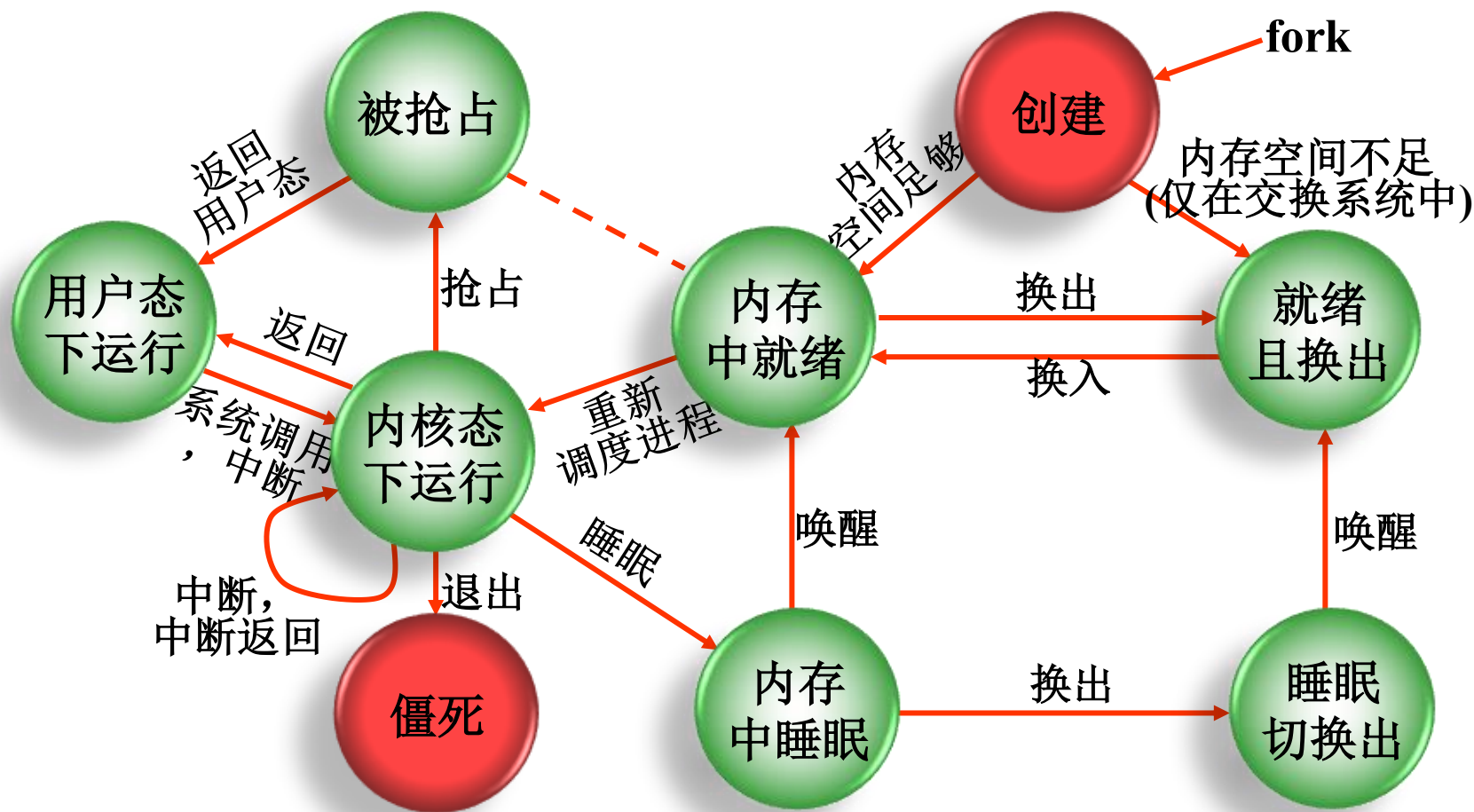
- 进程表项（**proc**结构）：进程的基本常用信息，常驻内存
- **U**用户区（**user**结构）：可以调出到外存，进程处于执行状态时才用得到，各种资源表格
- 本进程区表：定义了从虚地址到实地址的映射
- 核心栈：核心态执行时的过程调用的栈结构



3.3 进程控制

3.3.3 Unix的进程管理

(2) UNIX的进程状态及其转换





3.3 进程控制



3.3.3 Unix的进程管理

(2) UNIX的进程状态

进程状态	说明
用户态运行	在用户模式下运行
内核态运行	在内核模式下运行
就绪，并驻留在内存中	只要在内核调度到就立即执行
睡眠，并驻留在内存中	在某事件发生前不能执行，在内存
就绪，且换出	进程就绪，但被交换在外存，
睡眠，且换出	进程在等待一个事件，在外存
被抢占	进程从内核态转到用户态，但是内核抢占它，并作了进程切换
创建	进程刚被创建，还没做好执行的准备
僵死	进程不再存在，但是它留下一个记录，可由其父进程收集



3.3 进程控制

3.3.3 Unix的进程管理

(3) UNIX的进程控制

● UNIX进程控制的系统调用

- ① **fork()**: 创建一个新进程
- ② **exec()**: 执行一个可执行程序（加载）
- ③ **exit()**: 退出
- ④ **sleep()**: 暂停一段时间
- ⑤ **pause()**: 暂停并等待信号
- ⑥ **wait()**: 等待子进程暂停或终止
- ⑦ **kill()**: 发送信号到某个或一组进程
- ⑧ **ptrace()**: 设置执行断点(breakpoint), 允许父进程控制子进程的运行, gdb利用该调用实现单步调试



3.3 进程控制



● 创建原语描述（回顾原理）

Procedure Create(S0, P0, M0, R0)

Begin

i := getEmptyPCB(); //查找PCB集合，返回PCB的内部标识符 i

i.id := getpid; i.priority := P0; /* 初始化PCB */

i.CPU_State := S0;

i.Mainstore := M0;

i.resources := R0;

i.status := Ready;

j := EP; i.parent := j; /* 插入家族队列 */

i.progeny := $\wedge$; j.progeny := i;

i.sdata := RQ; insert(RQ, i); /* 插入就绪队列 */

End;



3.3 进程控制



3.3.3 Unix的进程管理

(3) UNIX的进程控制

🌐 **fork()—创建一个新进程**

- **fork()调用格式:** `pid = fork()`
- **精确复制:** `fork`创建一个子进程, 除了子进程标识符和其`proc`结构中的某些特性参数不同之外, 子进程是父进程的**精确复制**;
- **一次调用两次返回:** 在子进程中返回时, `pid`为0; 在父进程中返回时, `pid`为所创建的子进程的标识;
- **创建后的调度:** `fork()`创建子进程后, 执行**返回父进程**, 或**调度切换**到子进程以及其他进程;
- **返回后的执行点:** **子进程**是从`fork`调用返回时在用户态开始运行的。**父进程**的返回点与子进程的起点是相同的。
- **问题: 创建效率低。**现代Unix中引入了写时复制**COW**技术。



3.3 进程控制--一次调用两次返回

```
int global;  
main() {  
    int local;
```

父进程

```
    if ((child=fork()) == -1) {  
        创建失败  
    }
```

```
    if (child == 0) { //子进程分支
```

```
        if (execlp("B",...) == -1) {  
            加载程序失败  
        }
```

```
        global=local + 2;  
        exit();  
    }
```

父进程

```
    global=local + 1;  
    exit();  
}
```

```
int global;  
main() {  
    int local;
```

子进程

```
    if ((child=fork()) == -1) {  
        创建失败  
    }
```

```
    if (child == 0) {
```

子进程

```
        if (execlp("B",...) == -1) {  
            加载程序失败  
        }
```

```
        global=local + 2;  
        exit();  
    }
```

父进程

```
    global=local + 1;  
    exit();  
}
```



3.3 进程控制



3.3.3 Unix的进程管理

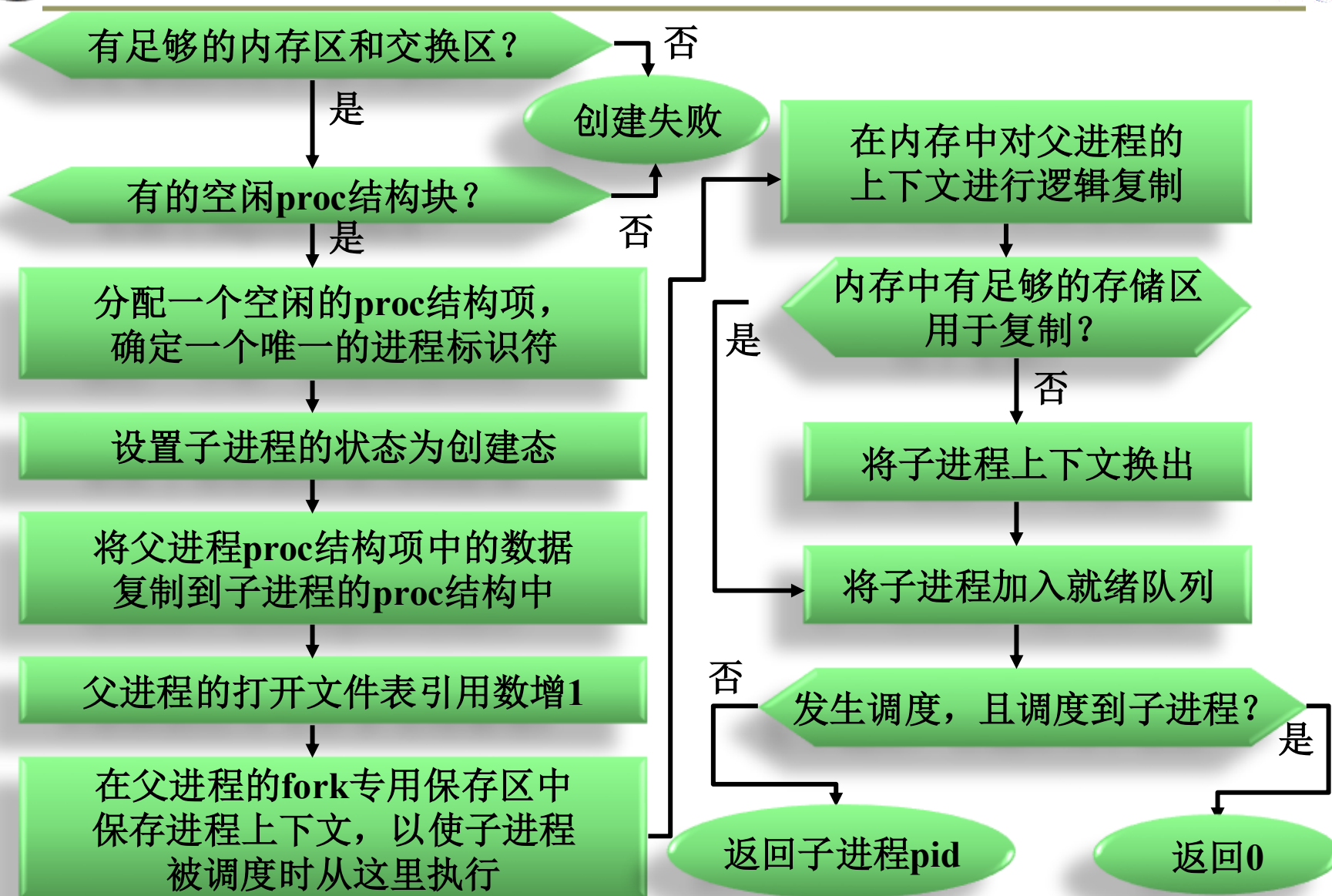
(3) UNIX的进程控制

● 创建进程的功能与处理流程

- ① 为子进程在proc结构表中分配一个空proc结构;
- ② 为进程赋予一个唯一的进程标识符pid;
- ③ 复制一个父进程上下文的逻辑副本。父子进程共享部分只改变共享区的引用数,而不复制;只复制父进程的非共享区部分;
- ④ 增加与父进程相关联的有关文件系统的进程引用计数;
- ⑤ 对于父进程,返回子进程的进程标识符;对于子进程,返回0。



3.3 进程控制





3.3 进程控制



3.3.3 Unix的进程管理

(3) UNIX的进程控制

● **exec()—执行一个文件的调用**

- ✓ 子进程如何执行一个新的程序文本？
 - 通过exec族调用，加载新的程序文本
- ✓ exec用一个新进程覆盖调用进程。
- ✓ 它的参数包括新进程对应的文件和命令行参数。
成功调用时，不再返回；否则，返回出错原因。
- ✓ 六种exec调用格式：各种调用的区别在于参数的
处理方法不同，常用的格式有：
 - `execvp(filename, argp)`
 - `execlp( filename,arg0,arg1,...,(char *)0)`



3.3 进程控制



例：UNIX进程创建

Program A

```
int global;
main() {
    int local;

    if ((child=fork()) == -1) {
        创建失败
    }
    if (child == 0) {
        子进程

        global=local + 2;
        exit();
    }
    父进程
    global=local + 1;
    exit();
}
```

```
if (execlp("B",...) == -1) {
    加载程序失败
}
```

Program B

```
main() {
    exit();
}
```

子进程被创建后，使用execlp系统调用, 用一个程序取代原来内存空间中的内容，然后开始执行。此后，两个进程各行其道。



3.3 进程控制

3.3.4 现代的进程创建机制（Linux/openEuler）

(1) 传统Unix的进程创建

❖ 传统的UNIX OS采用统一的方式fork()来创建进程

➤ 子进程复制父进程所拥有的资源

❖ 缺点：

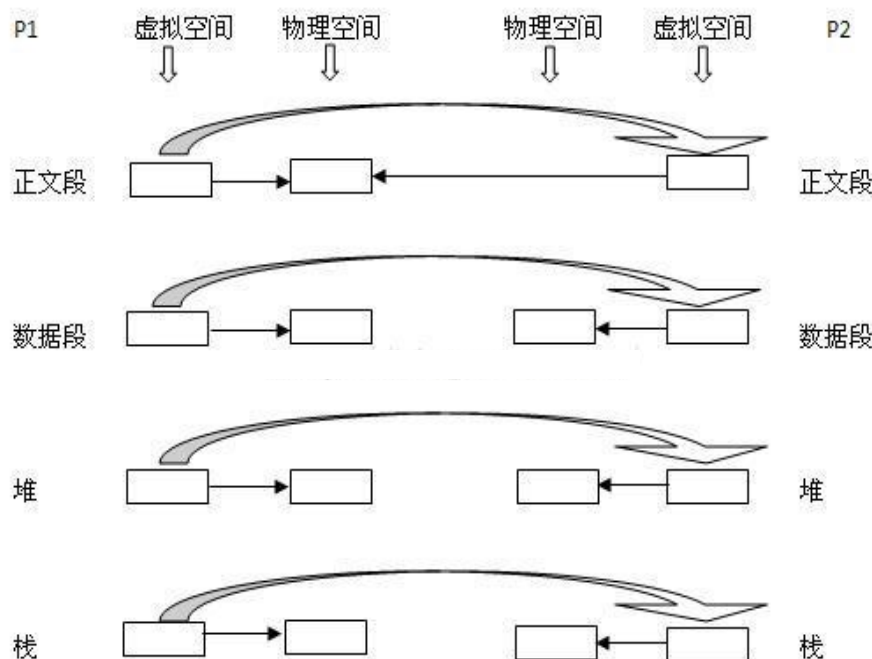
➤ 创建过程慢、效率低

➤ 原因：子进程复制的很多资源是不会使用到的

❖ 问题：

❖ 先复制有必要？

❖ 如何改进呢？





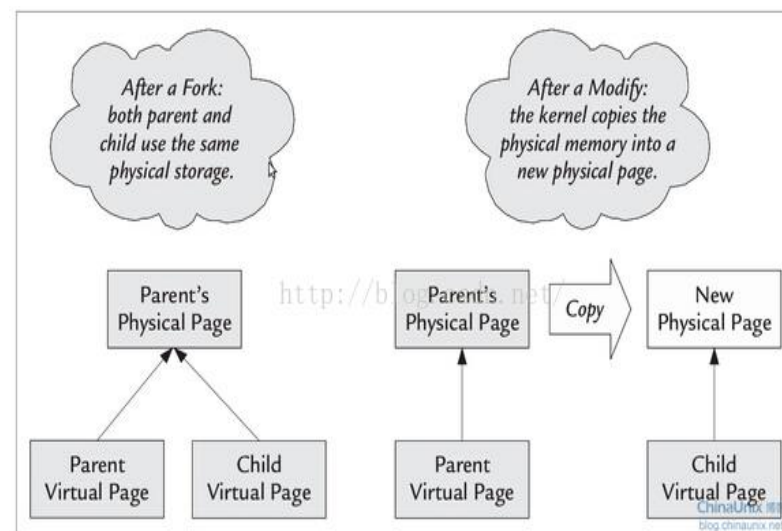
3.3 进程控制

3.3.4 现代的进程创建机制（Linux/openEuler）

（2）现代Unix/Linux的进程创建

❖ 现代UNIX内核引入三种不同的机制：**COW, clone, vfork**

➤ **写时复制技术(Copy-On-Write, COW)**: 执行fork()后, 允许父子进程共享相同的物理页帧, 只有当两者有一个进程试图写一个物理页, 内核才把这个页面的内容拷贝到一个新物理页, 并把这个新的物理页分配给正在写的进程。



- **clone（轻量级进程）** 允许父子进程共享许多数据结构：页表、打开的文件列表、信号处理例程等。
- **vfork**: 使用vfork创建的新进程能够共享父进程的内存地址空间，父进程在这个过程中被阻塞，直到子进程退出或者执行一个新的程序

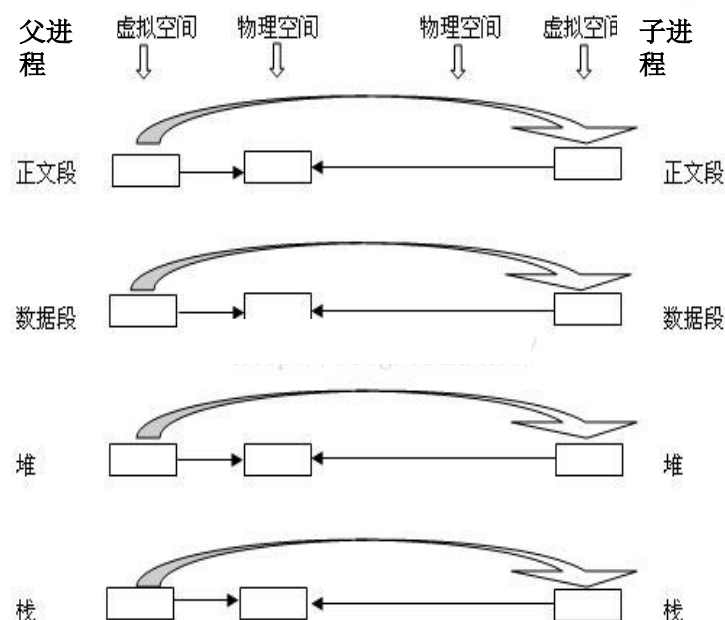


3.3 进程控制

3.3.4 现代的进程创建机制（Linux/openEuler）

（2）现代Unix/Linux的进程创建

- 原型：int fork(void)
- 使用fork()创建进程的特点
 - 调用一次，返回两次
 - 采用COW技术，，推迟甚至避免了大量的物理数据复制，父子进程共享物理空间，但子进程有自己的虚拟地址空间，父子进程具有良好的并发性。加快了执行速度。
- 思考：虚拟空间怎么复制？





3.3 进程控制



思考

1. **Unix/Linux**为什么选择使用**fork+exec**的方式创建进程？有什么益处？



3.3 进程控制

小结

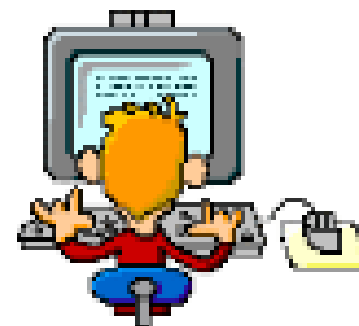
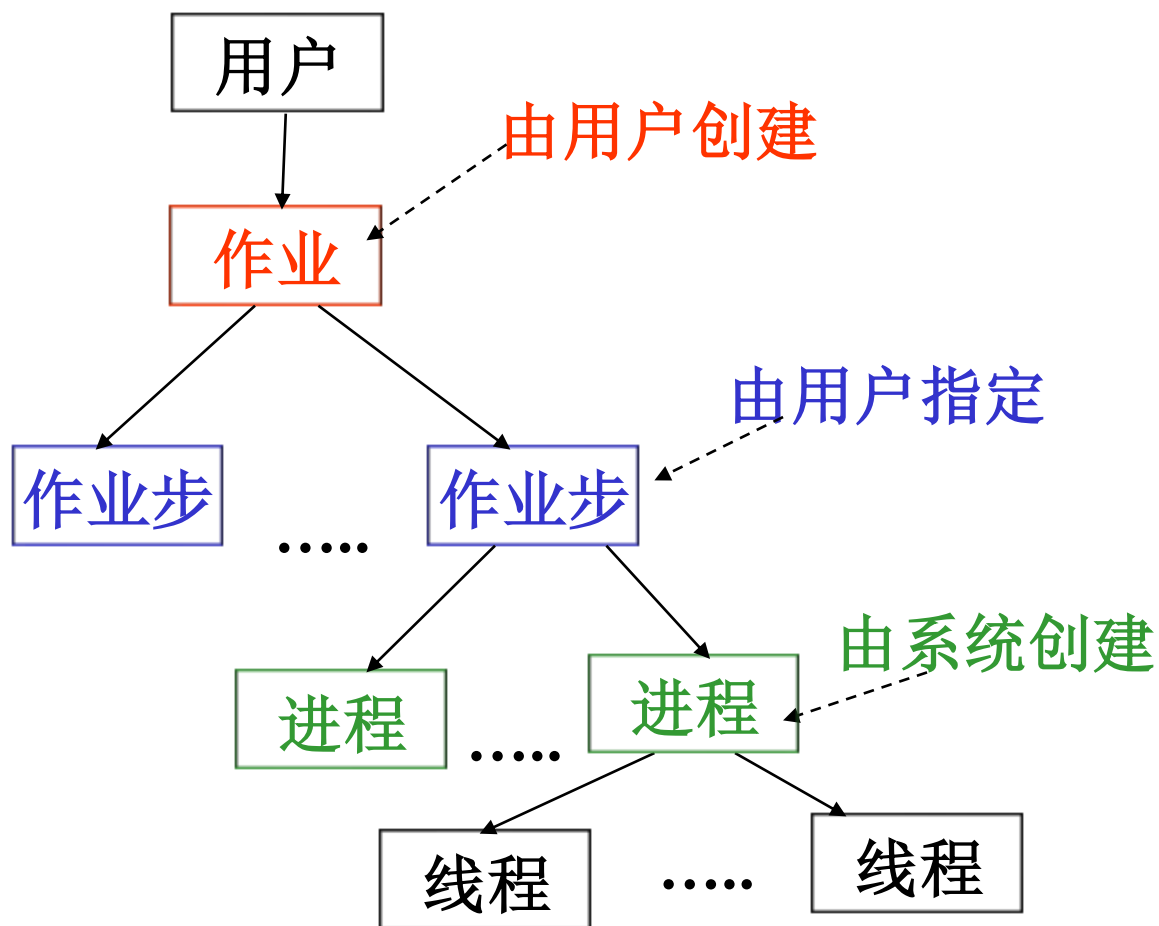
- 原语
- 进程控制原语：创建、撤销、阻塞与唤醒、挂起与激活
- UNIX的进程、状态、控制原语（**fork**）
- COW技术
- 由**fork()**的现代实现想到的：创新/革新的无处不在；计算机科学家对高效目标的不懈追求



回顾



作业、进程与程序的关系





Review



- 原语
- 进程控制原语
 - 创建与撤销
 - 阻塞与唤醒
 - 挂起与激活
- UNIX的进程、状态、控制原语（Fork）
- 一组测试题



第三章 进程管理



为了描述程序在并发执行时对系统资源的共享，需要一个描述程序执行时动态特征的概念，这就是进程。本章将讨论进程概念、进程控制和进程间关系。

1. 概述
2. 进程的描述
3. 进程控制
4. 线程
5. 进程互斥和同步
6. 进程间通信
7. 死锁问题
8. 进程其他方面的举例



第三章 进程管理

1. 概述
2. 进程的描述
3. 进程控制

4. 线程

- ① 线程的引入
- ② 线程的优点
- ③ 线程的属性
- ④ 与进程比较
- ⑤ 与进程的关系
- ⑥ 线程的状态
- ⑦ 线程的实现方式
- ⑧ 线程库及其他

Objectives:

1. 掌握线程的概念、与进程的联系与区别;
2. 了解线程状态与转换;
3. 了解线程的基本实现原理和实现方式;
4. 能够**利用**线程库pthread,
设计多线程程序



3.4 线程(THREAD)

● 回顾进程：

✓ 在传统的OS中，进程体现两个属性：

① 资源分配的单位

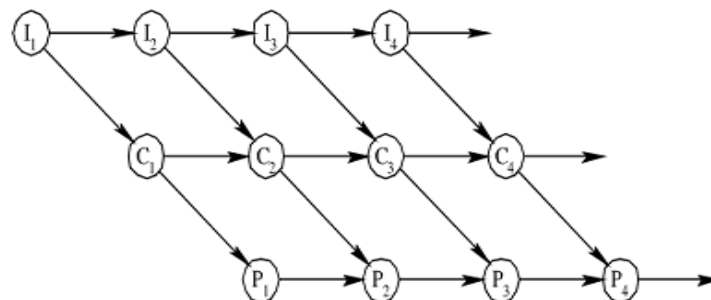
② 调度运行的单位

✓ 引入进程：

➤ 描述程序的动态执行过程及其特性；

➤ 多道程序并发执行，提高资源的利用率和系统吞吐量；

✓ 问题：程序执行的并发度是否可以再提升？





3.4 线程(THREAD)



本节内容包括:

- 为什么要引入线程? -->线程的引入
- 进程和线程有什么区别? -->进程和线程的比较
- 线程如何实现? -->线程的实现方式
- 典型操作系统的线程实现-->线程举例



3.4 线程(THREAD)



3.4.1 线程的引入

- 引入**进程**的目的:

- 使多个程序能并发执行,
- 提高**资源利用率和系统吞吐量**;

- 引入**线程**的目的:

- 减少程序在并发执行时所付出的**时空开销**, 使OS具有更好的并发性。
 - 进程的**创建** (建立上下文)、**撤销** (回收上下文)、**切换** (保存上下文) 的**开销大**
 - 例如, **Web服务器**、**数据库服务器**、**远程数据库访问**等。



3.4 线程(THREAD)

3.4.1 线程的引入

● 通过线程可方便有效地实现并行性

- 当一个应用是由若干个相对独立的任务构成时特别有用。例如读数据、处理数据、输出数据
- 进程可创建多个线程来执行同一程序的不同部分
- 例如：Web服务器、RPC、数据库服务器

● 创建线程快，开销少

- 一个进程创建的所有线程可共享进程的资源，不需要特殊的通信机制。如：一个线程将输出写入某数据区，另一线程可以读出作为输入；
- 线程之间切换的开销要比进程之间切换的开销小；

● 多线程机制

- 对多个客户同时提出服务请求时的回答十分有利。



3.4 线程(THREAD)



3.4.1 线程的引入

● **进程模型**在处理“**基于同数据区的同时多请求**”时的效率局限性，

➤ 例：**售票系统**：数据库服务器软件需同时处理来自多个用户进程的读盘请求，这些请求都是针对同一个盘，可有如下几种方式：

- **进程**：一个进程来顺序处理
- **多个相互独立的进程**：每个进程负责处理一个请求
- 用一个**进程来并发**处理所有请求（读盘时要阻塞，处理下一个请求）——**线程**



3.4 线程(THREAD)



3.4.1 线程的引入

多线程的适用场景

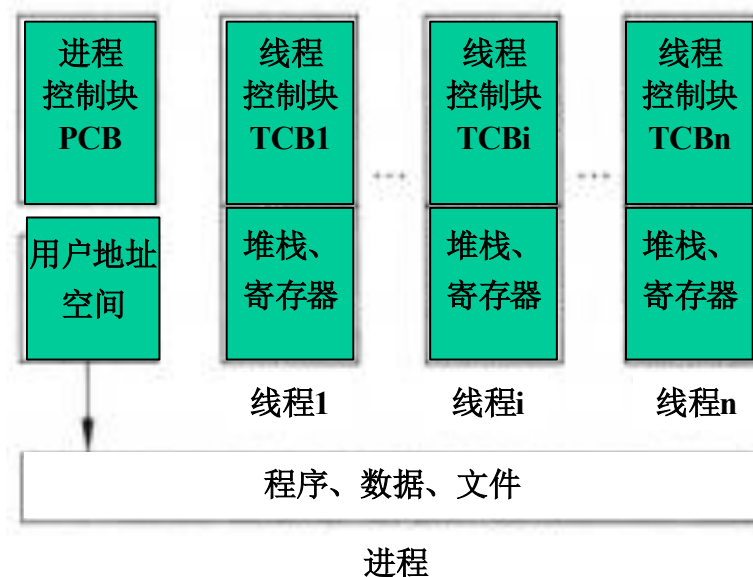
- 1、常见的**浏览器、Web服务**(现在写的web是中间件帮你完成了线程的控制), web处理请求, 各种专用服务器(如游戏服务器)
- 2、**servlet多线程**
- 3、**FTP下载**, 多线程操作文件
- 4、数据库用到的多线程
- 5、**分布式计算**
- 6、**tomcat**, tomcat内部采用多线程, 上百个客户端访问同一个WEB应用, tomcat接入后就是把后续的处理扔给一个新的线程来处理, 这个新的线程最后调用servlet程序, 比如doGet或者doPost方法
- 7、**后台任务**: 如定时向大量(100W以上)的用户发送邮件; 定期更新配置文件、任务调度(如quartz), 一些监控用于定期信息采集
- 8、**自动作业处理**: 比如定期备份日志、定期备份数据库
- 9、**异步处理**: 如发微博、记录日志
- 10、**页面异步处理**: 比如大批量数据的核对工作(有10万个手机号码, 核对哪些是已有用户)
- 11、**数据库的数据分析**(待分析的数据太多), 数据迁移
- 12、**多步骤的任务处理**, 可根据步骤特征选用不同个数和特征的线程来协作处理, 多任务的分割, 由一个主线程分割给多个线程完成
- 13、**desktop应用开发**, 一个费时的计算开个线程, 前台加个进度条显示



3.4 线程(THREAD)

3.4.1 线程的引入--定义

- **线程** (thread) 是指进程中一个单一顺序的控制流，一个进程中可以并发多个线程，每条线程并行执行不同的任务。
- **线程是**独立调度和分派的基本单位，是OS能够进行运算调度的最小单位，是进程中的实际运作单位。
- 同一进程中的多条线程将**共享**该进程中的全部系统资源，如**虚拟地址空间，文件描述符和信号处理等**。
- 同一进程中的多个线程有各自的调用栈 (call stack)、寄存器上下文、线程本地存储 (thread-local storage) 。





3.4 线程(THREAD)



3.4.2 线程的优点

- **创建和撤销快**：线程的创建时间、终止时间比进程短；
- **线程切换快**：同进程内的线程切换时间比进程短（上下文有许多是相同的）；
- **同进程内线程间通信快且高效**：由于同进程内线程间共享进程的代码、数据、内存和文件资源，可直接进行不通过内核的**通信；快且高效**
- **线程并发执行的时间和空间开销**（线程的创建、退出和调度）**减小**。因此允许在系统中建立更多的线程来提高并发程度；
- 一个进程的所有线程必须同时处于**挂起状态**；进程的终止会导致所有线程的终止。---与**线程的实现方式有关**



3.4 线程(THREAD)



3.4.3 线程的属性 (1/2)

- 在传统的OS中，进程体现两个属性
 - 资源分配的单位
 - 调度运行的单位
- 进程控制和调度的时空代价大：实施进程的创建、删除和切换过程中时空代价大，限制了系统中的进程数目和并发活动的程度
- 将两个属性分开处理
 - 线程：对于作为调度和分派的基本单位，不同时作为拥有资源的单位，以做到“轻装上阵”
 - 进程：对于拥有资源的基本单位，不对之进行频繁的切换

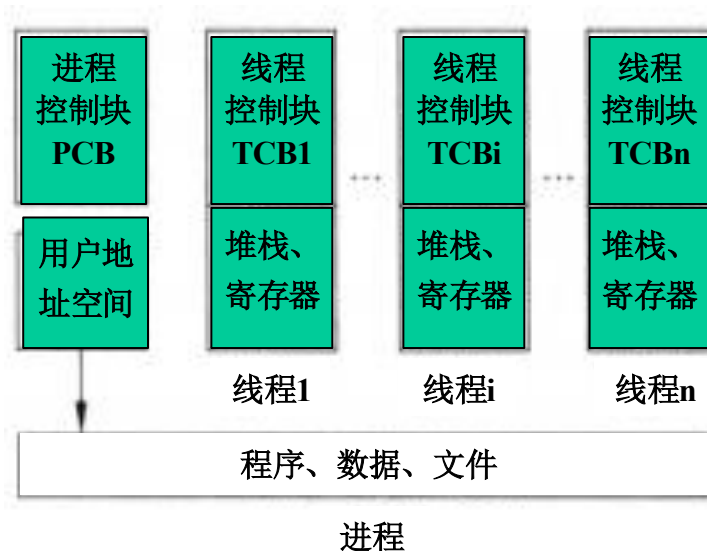


3.4 线程(THREAD)



3.4.3 线程的属性 (2/2)

- **轻型实体**: 线程中的实体一般不拥有系统资源, 只是有一点必不可少的、能保证其独立运行的资源, 如TCB、寄存器和堆栈;
- **可并发执行**: 不同进程的线程和一个进程中的线程;
- **共享进程资源**: 同一进程中的各线程可共享该进程所拥有的资源。如具有相同的进程地址空间、访问进程所拥有的已打开文件等。





3.4 线程(THREAD)

3.4.4 进程与线程的比较 (1/4)

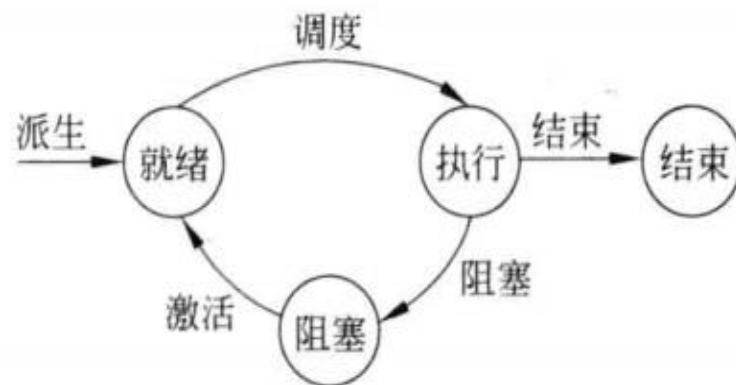
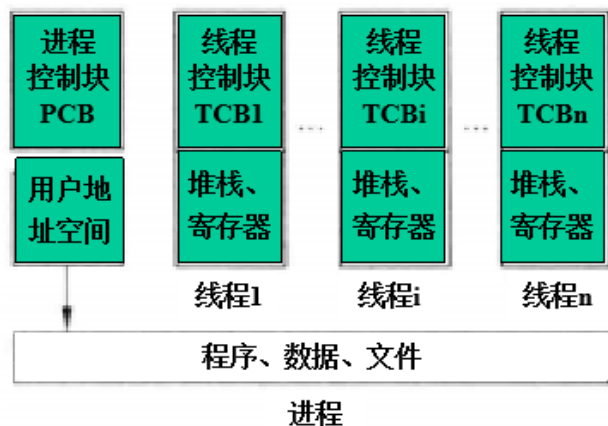
● **进程Process:** 资源分配单位（存储器、文件），又称为“任务（task）”

✓ 多线程是指OS支持在一个进程中执行多个线程的能力

● **线程Thread:** 作为CPU调度单位

✓ 只拥有必不可少的资源，如：TCB、寄存器、栈

✓ 同样具有就绪、阻塞和执行三种基本状态





3.4 线程(THREAD)



3.4.4 进程与线程的比较 (2/4)

● 调度

- 在无线程的OS中，**进程**作为拥有资源的基本单位和独立调度、分派的基本单位；
- 引入线程的OS中，**线程**作为调度、分派的基本单位；**进程**作为拥有资源的基本单位；
- **关于进程的切换：**
 - ✓ 同一进程中，线程的切换不会引起进程的切换；
 - ✓ 从一个进程中的线程切换到另一个进程中的线程时，将会引起进程的切换。

- **并行性：**在引入线程的OS中，不仅**进程之间可并发**，在一个进程中的**多个线程间也可并发**执行，从而提高系统资源的**利用率和系统吞吐量**。



3.4 线程(THREAD)



3.4.4 进程与线程的比较 (3/4)

● 拥有资源

- 进程是系统中拥有资源的基本单位;
- 一般而言, 线程不拥有系统资源 (除TCB、栈等), 但它可以访问其隶属进程的资源, 如一个进程的代码段、数据段等。

● 系统开销

- 线程的创建、撤销、切换快, OS所付出的**开销明显小于进程**;
- **通信方面**:
 - ✓ 由于同进程内**线程间**共享进程的代码、数据、内存和文件资源, 可直接进行不通过内核的通信;
 - ✓ 而**进程间**的通信需要通过内核进行, 以提供保护和通信所需机制。



3.4 线程(THREAD)



3.4.4 进程和线程的比较 (4/4)

每个线程的内容

程序计数器

堆栈

寄存器组

子线程

状态

每个进程的内容

地址空间

全局变量

打开文件

子进程

定时器

信号

信号量

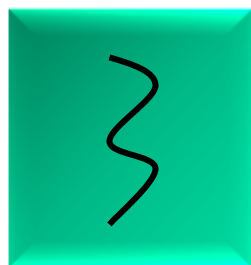
记帐信息



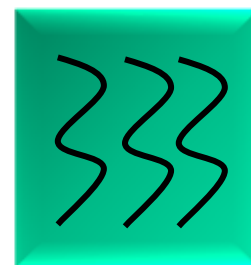
3.4 线程(THREAD)



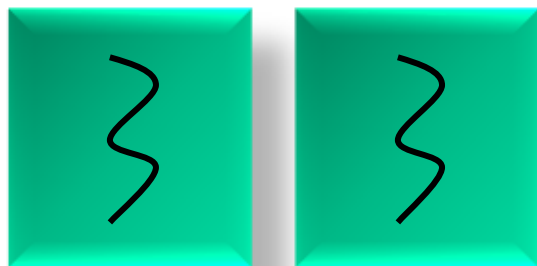
3.4.5 进程与线程的关系 (1/2)



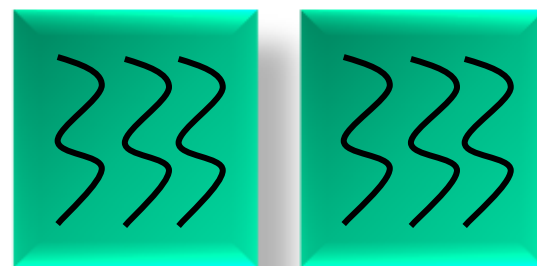
**one process
one thread**



**one process
multiple threads**



**multiple processes
one thread per process**

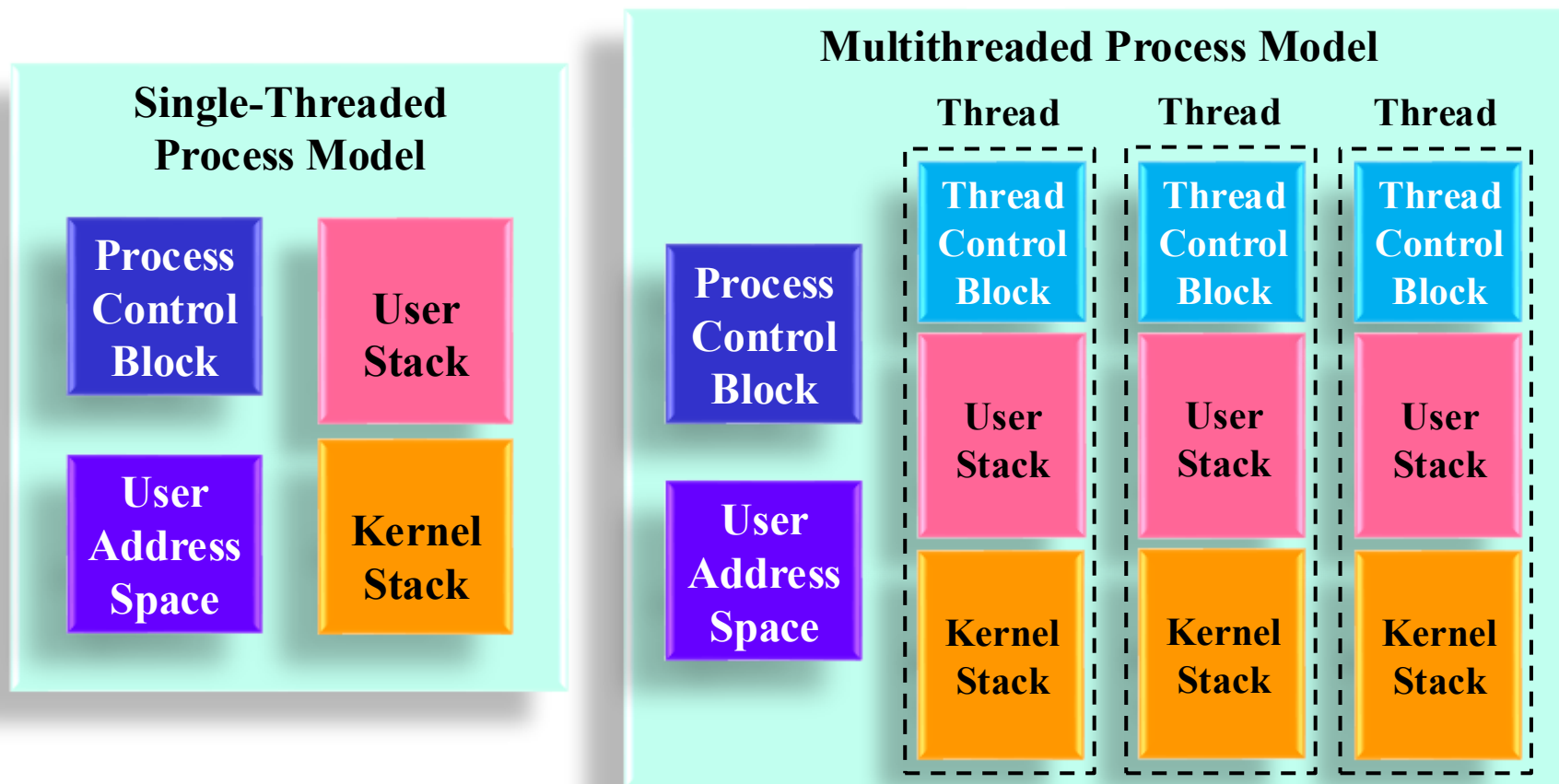


**multiple processes
multiple threads per process**



3.4 线程(THREAD)

3.4.5 进程与线程的关系 (2/2)



线程组成: TCB、堆栈、寄存器



3.4 线程(THREAD)



3.4.6 线程的适用范围

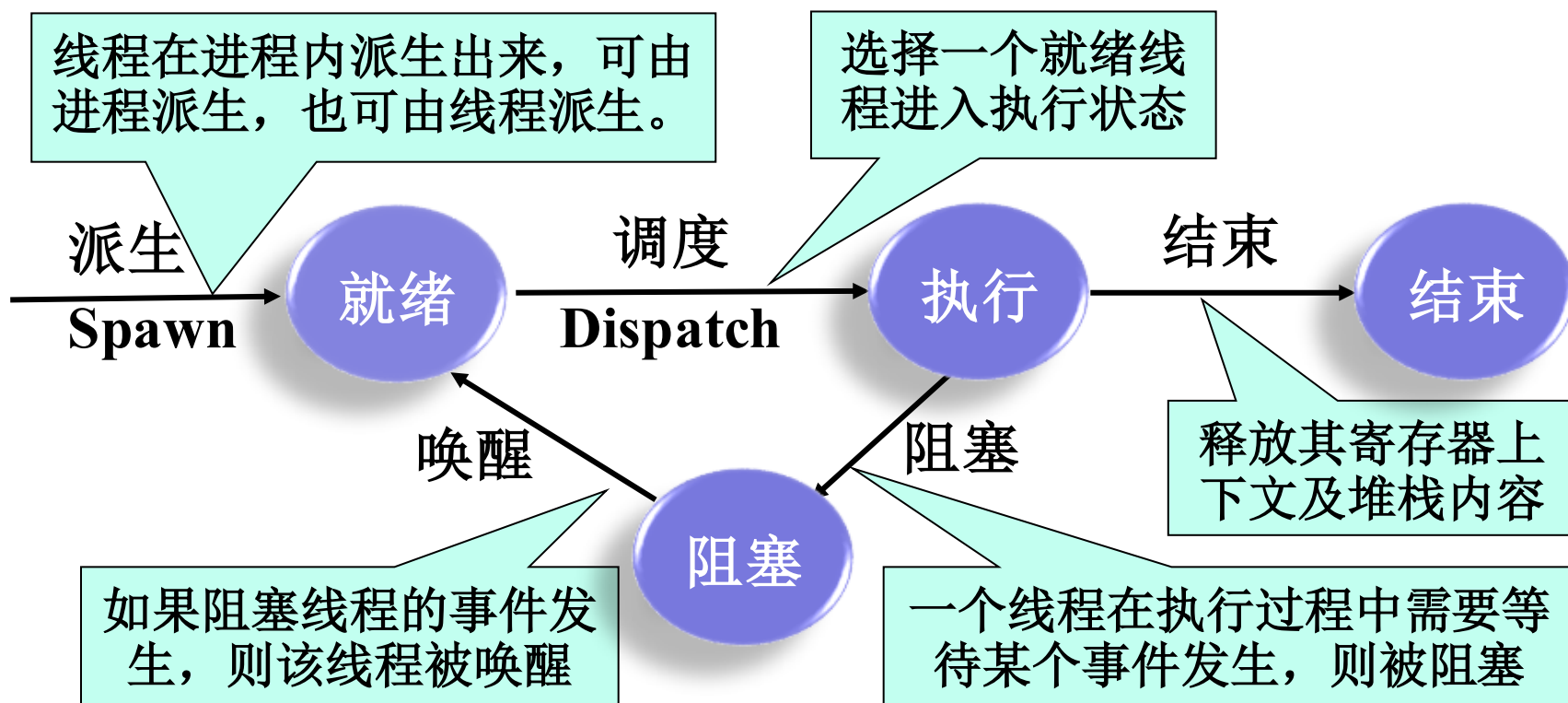
- **不适宜**：很少做进程调度和切换的实时系统、个人数字助理系统。任务单一；
- **最适宜**：多处理机系统、大并发的场景；
- **典型应用环境**
 - ✓ **服务器程序**：服务器中的文件管理或通信控制或客户服务器方式的服务器程序；
 - ✓ **前后台处理**：如表处理程序，用户输入命令和命令执行；
 - ✓ **异步处理**：一个程序中可以并发的程序段，使用不同的线程。



3.4 线程(THREAD)

3.4.7 线程的状态

● 基本状态：执行、就绪、阻塞





3.4 线程(THREAD)



3.4.8 OS对线程的实现方式

- (1) 内核级线程 (**KLT, Kernel-level thread**)
- (2) 用户级线程 (**ULT, User-level thread**)
- (3) 轻权进程 (**Light Weight Process**)
- (4) 混合线程 (**Hybrid thread**)



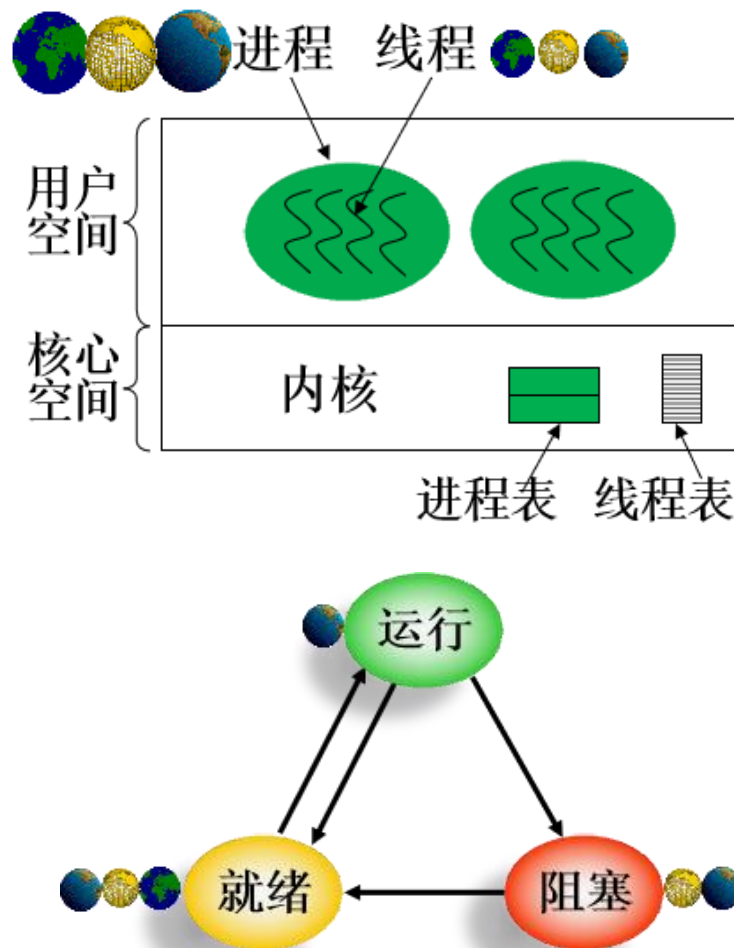
3.4 线程(THREAD)



3.4.8 OS对线程的实现方式

(1) 内核级线程 (KLT)

- ✓ 线程在内核空间实现，核心知道线程存在；
- ✓ 内核维护进程和线程的上下文信息，由内核完成线程切换；
- ✓ 一个线程发起系统调用而阻塞，不会影响其他线程的运行；
- ✓ 时间片分配给线程，所以多线程的进程获得更多CPU时间。
- ✓ 如：Windows NT、Linux支持KLT；
- ✓ **缺点：**对用户线程切换而言，需要从用户态到内核态的切换，模式切换开销大，





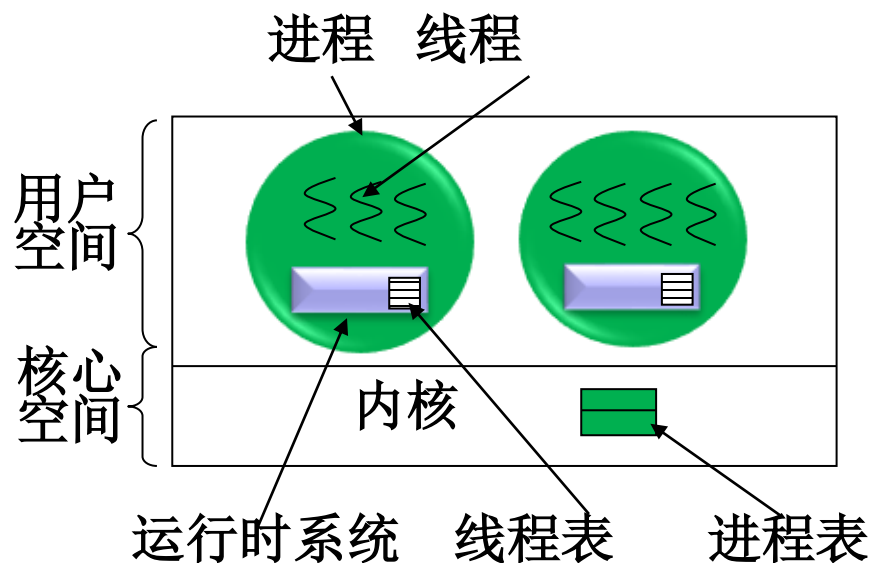
3.4 线程(THREAD)



3.4.8 OS对线程的实现方式

(2) 用户级线程 (User-level thread)

- 把**线程库**整个放在用户空间，核心只对**常规进程**进行管理，并**不知道**线程是否存在；
- 用户线程的维护由**应用进程**完成，用户线程切换不需内核特权





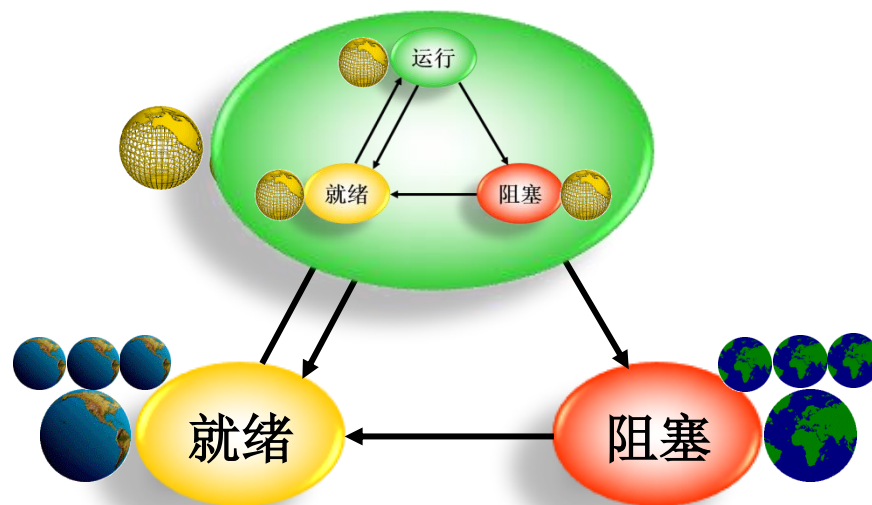
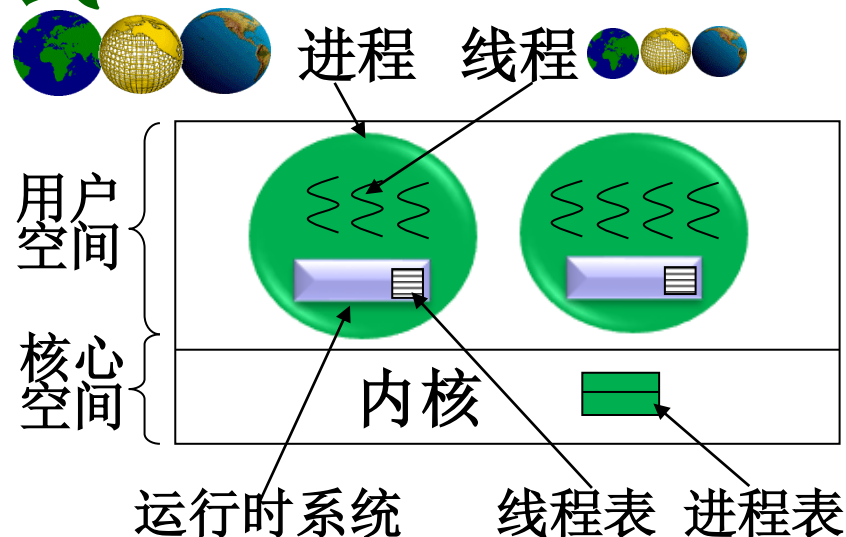
3.4 线程(THREAD)



3.4.8 OS对线程的实现方式

(2) 用户级线程 (ULT)

- **应用进程**利用线程库提供创建、同步、调度和管理线程的函数来控制用户线程，不依赖于OS核心。
- **调度由应用软件内部进行**，通常采用非抢先式和更简单的规则，无需用户态/核心态切换。
- **时间片分配给进程**，多线程则每个线程就慢。OS的调度单位仍是进程。如果进程的调度时间片为T，则在T内，用户可以根据自己的需要设置不同的线程调度算法。
- **一个线程**发起系统调用而**阻塞**，则**整个进程**在等待。



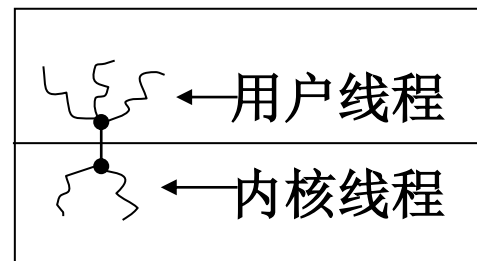
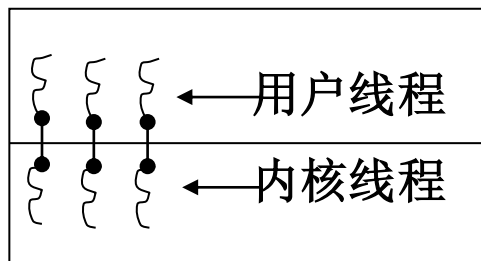
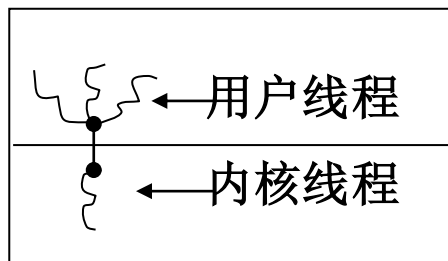


3.4 线程(THREAD)

3.4.8 OS对线程的实现方式

(3) 轻权进程 (Light Weight Process)

- 它是内核支持的**用户线程**。一个进程可有一个或多个轻权进程，每个轻权进程支持一个或多个用户线程，并映射到一个内核线程。
- 由于同时提供**内核线程控制机制**和**用户线程库**，因此可很好地把**内核线程**和**用户线程**的优点结合起来。



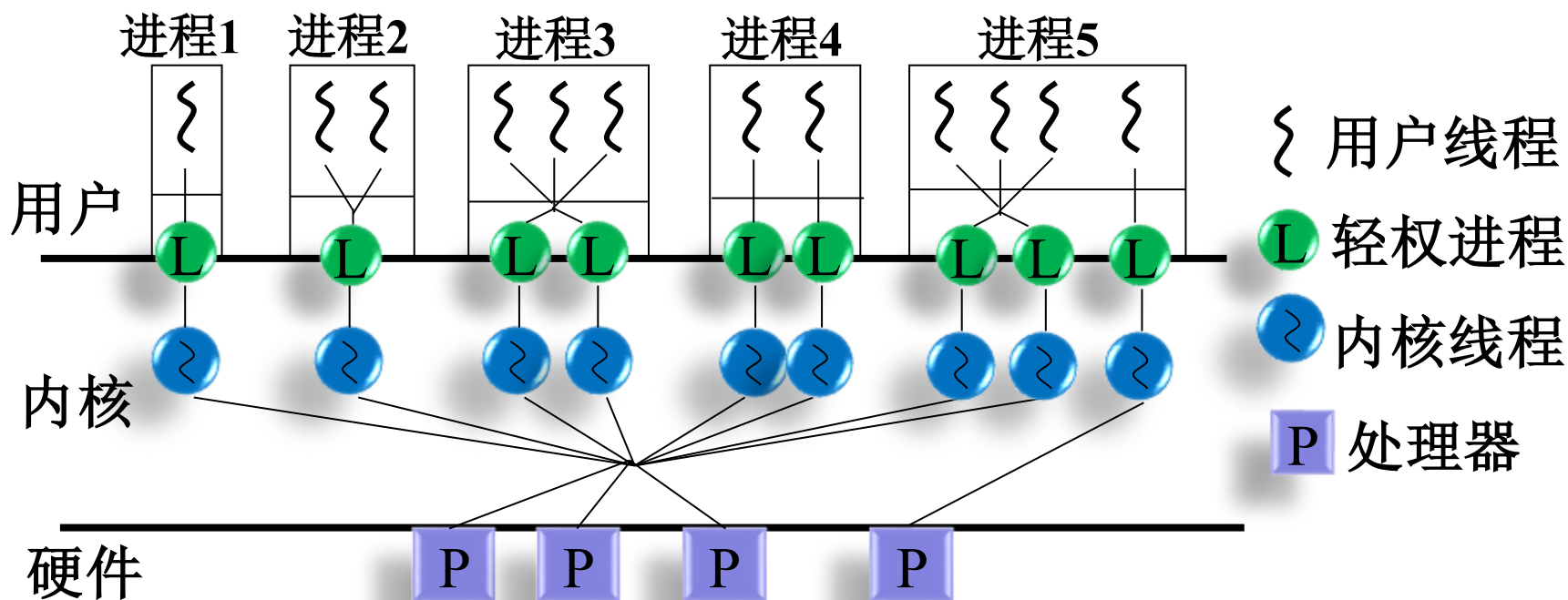


3.4 线程(THREAD)



3.4.8 OS对线程的实现方式

- Solaris系统采用混合线程 (hybrid thread)





3.4 线程(THREAD)



3.4.9 线程库举例——Pthread

- 线程库
 - 编程语言/OS为程序员提供的，用于创建、管理线程的API集合
- Pthread
 - UNIX家族的标准多线程库
 - Pthread只定义接口，是ULT还是KLT取决于OS实现



3.4 线程(THREAD)



3.4.9 线程举例 (NT_thread.cpp)

```
#include <stdio.h>
#include <windows.h>
#include <iostream.h>
#include <winbase.h>
void SubThread(void)
{   int i;
    for (i=0;i<5;i++) {
        cout << "SubThread" << i << endl;
        sleep(2000);
    }
}
```



3.4 线程(THREAD)



```
void main(void)
{
    cout << "CreateThread" << endl;
    // Create a thread
    DWORD IDThread;
    HANDLE hThread;
    hThread = CreateThread(
        NULL,                // no security attributes
        0,                   // use default stack size
        (LPTHREAD_START_ROUTINE) SubThread, // thread function
        NULL,                // no thread function argument
        0,                   // use default creation flags
        &IDThread);          // returns thread identifier
    // Check the return value for success
    if (hThread == NULL)
        cout << "CreateThread error" << endl;
```



3.4 线程(THREAD)



```
int i;
for (i=0;i<5;i++) {
    cout << "MainThread" << i << endl;
    if (i==1) {
        if (SuspendThread(hThread)==0xFFFFFFFF) {
            cout << "Suspend thread error." << endl; }
        else {
            cout << "Suspend thread is ok." << endl; }
    }
    if (i==3) {
        if (ResumeThread(hThread)==0xFFFFFFFF) {
            cout << "Resume thread error." << endl; }
        else {
            cout << "Resume thread is ok." << endl;}
    }
    sleep(4000);
} //end for
}
```



3.4 线程(THREAD)

3.4.9 线程举例 (NT_thread.cpp)

Create Thread

MainThread0

SubThread0

SubThread1

MainThread1

运行结果 → Suspend thread is ok.

MainThread2

MainThread3

Resume thread is ok.

SubThread2

SubThread3

MainThread4

SubThread4



3.4 线程(THREAD)



• Linux的线程创建实例:

(1) **pthread_create**函数: UNIX/Linux创建线程

- ✓ 头文件: `#include<pthread.h>`
- ✓ 函数声明: `int pthread_create(pthread_t *restrict tidp, const pthread_attr_t *restrict attr, void* (*start_rtn)(void*), void *restrict arg);`
- ✓ 返回值: 成功则返回0, 否则返回出错编号
- ✓ 参数:
 - 第一个参数为指向线程标识符的指针。
 - 第二个参数用来设置线程属性。
 - 第三个参数是线程运行函数的地址。
 - 最后一个参数是运行函数的参数。
- ✓ 注意: 在编译时注意加上**-l pthread**参数, 以调用静态链接库。因为pthread并非Linux系统的默认库。

(2) **pthread_join**函数: 用来等待一个线程的结束。

- ✓ `extern int pthread_join __P (pthread_t __th, void ** __thread_return);`
- ✓ 参数: 第一个参数为被等待的线程标识符; 第二个参数为一个用户定义的指针, 它可以用来存储被等待线程的返回值。
- ✓ 注意: 这个函数是一个线程阻塞的函数, 调用它的函数将一直等待到被等待的线程结束为止, 当函数返回时, 被等待线程的资源被收回。如果执行成功, 将返回0, 如果失败则返回一个错误号。

实例见文件: [第三章-进程管理-线程-3-实例1.docx](#)



线程库举例——Pthread



```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

子线程ID, 指针

父线程创建子线程, 子线程执行runner函数

父线程等待, 直至子线程返回

子线程退出



线程库举例——Windows Threads



```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
```

Windows操作系统 采用的是内核 级线程实现

父线程创建子线程

父线程等待，直至子线程返回

父线程清理子线程资源



线程库举例——Java Threads



- 由于Java使用JVM虚拟机运行时环境，因此多线程在JVM中具体实现
- Java提供了很多种多线程实现方法（下面的例子只是其中一种）
- JVM可运行在不同操作系统上，所以ULT或KLT取决于底层操作系统

```
import java.util.concurrent.*;

class Summation implements Callable<Integer>
{
    private int upper;
    public Summation(int upper) {
        this.upper = upper;
    }

    /* The thread will execute in this method */
    public Integer call() {
        int sum = 0;
        for (int i = 1; i <= upper; i++)
            sum += i;

        return new Integer(sum);
    }
}
```

```
public class Driver
{
    public static void main(String[] args) {
        int upper = Integer.parseInt(args[0]);
```

```
        ExecutorService pool = Executors.newSingleThreadExecutor();
        Future<Integer> result = pool.submit(new Summation(upper));
```

```
        try {
            System.out.println("sum = " + result.get());
        } catch (InterruptedException | ExecutionException ie) { }
```

```
    }
```

父线程创建子线程，子线程启动

父线程等待子线程返回



3.4 线程(THREAD)



3.4.10 Linux(openEuler)的线程

- Linux的线程模型

- 早期的Linux线程就属于用户级线程模型。
- Linux 2.6内核使用的是内核级线程的实现方式，实现了轻量级进程（LWP, Light Weight Process）的概念，轻量级进程实质就是线程。
- 在Linux内核中，进程和线程不需要区分，内核使用统一的方式对它们进行创建、管理和调度。
- Linux的轻量级进程保留了线程的特性，例如，没有独立的地址空间、线程之间共享资源(共享同一内存地址空间、同一打开文件集)等。
- Linux以一种简单的方式实现了内核级线程。Linux中一个线程组基本上就是实现了多线程应用的一组轻量级进程。
- 系统调用：fork()、vfork()、clone()。



3.4 线程(THREAD)



3.4.10 Linux(openEuler)的线程

- **Linux的POSIX线程库**

- 对于Linux而言，没有**真的线程**，只是一种**轻量级的进程**。所以Linux的线程库并不是OS提供的，而是一个**第三方库**。
- 与线程有关的函数系列，大多数函数名字是以“**pthread_**”打头的。
- 要使用这些函数库，要通过引入**头文件**<pthread.h>
- 链接这些线程函数库时要使用**编译器命令**的“**-lpthread**”选项

- **Linux的POSIX线程库中的函数**

- **线程创建、终止**: pthread_create()、pthread_exit() ;
- **线程等待**: pthread_join()。调用该函数的线程将挂起等待,直到id为thread的线程终止;
- **分离线程**: pthread_detach()。当线程退出时,自动释放线程资源。这就是分离线程。
- **线程互斥**: pthread_mutex_init、pthread_mutex_destroy、pthread_mutex_lock、pthread_mutex_unlock
- **线程同步**: pthread_cond_init、pthread_cond_destroy、pthread_cond_wait、pthread_cond_broadcast、pthread_cond_signal



3.4 线程(THREAD)



3.4.10 Linux(openEuler)的线程

POSIX线程库中的函数索引:

- 1.创建线程pthread_create
- 2.等待线程结束pthread_join
- 3.分离线程pthread_detach
- 4.创建线程键pthread_key_create
- 5.删除线程键pthread_key_delete
- 6.设置线程数据pthread_setspecific
- 7.获取线程数据pthread_getspecific
- 8.获取线程标示符pthread_self
- 9.比较线程pthread_equal
- 10.一次执行pthread_once
- 11.出让执行权sched_yield
- 12.修改优先级pthread_setschedparam
- 13.获取优先级pthread_getschedparam
- 14.发送信号pthread_kill
- 15.设置线程掩码pthread_sigmask

- 16.终止线程pthread_exit
- 17.退出线程pthread_cancel
- 18.允许/禁止退出线程pthread_setcancelstate
- 19.设置退出类型pthread_setcanceltype
- 20.创建退出点pthread_testcancel
- 21.压入善后处理函数void
pthread_cleanup_push(void (*routine)(void
*), void *args);
- 22.弹出善后处理函数void
pthread_cleanup_pop(int execute);
- 23:初始化线程属性
pthread_attr_init(pthread_attr_t *tattr)
;
- 24.设置线程分离状态的函数为
pthread_attr_setdetachstate (
pthread_attr_t *attr, int detachstate) ;
- 25.去除线程属性的初始化 int
pthread_attr_destroy(pthread_attr_t
attr);

详见: pthread库函数, <https://blog.csdn.net/woshichaoa/article/details/11392433>



3.4 线程(THREAD)



- 小结
 - 引进线程的目的
 - 线程的属性
 - 线程的状态
 - 线程与进程的区别
 - 线程的创建方式：用户级、核心级、轻量进程
 - 多线程合作完成任务 <--> 多人合作完成任务
 - 各尽其责，互相合作，团结一致
 - 做好善后，留有余香



第三章 进程管理小结



- **进程概念**
 - 程序顺序执行、程序并发执行、引入进程
- **进程描述：PCB、PCB作用**
- **进程状态及转换：3、5、7、8**
- **PCB的组织：PCB表、按类型链表、索引表...**
- **进程控制：原语、创建vs撤销、阻塞vs唤醒、挂起vs激活**
- **线程：概念、实现方式**
- **线程实现举例**
- **参考：**
 - **B站，Linux系统程序设计-线程**
 - **Linux中pthread线程使用详解，
<https://blog.csdn.net/JoysonQin/article/details/70237422>**



3.4 线程--作业与思考

1. 题目：使用Pthread，或Windows Thread，或Java多线程编写一个小程序，用fork-join的方式实现矩阵乘法运算（eg. 20×16 , $16 \times 30 \rightarrow 20 \times 30$ ）。如果可能，给出两种不同数量并行线程的例子，猜测不同实现的性能差异，实际测量不同实现的性能差异，分析并给出原因。并设计实验验证是用户级线程还是核心级线程。

交付内容：

- 源代码、编译后的可执行文件
- 编译环境详细文档
- 运行的输入、输出界面
- 性能分析结果（如果有）
- 给出实验设计说明和结果